Nametable Mapping

File: nametable_mapping.kicad_sch

CPU Address Bus

File: cpuAddressBus.kicad_sch

Register Access

File: register_access.kicad_sch

PRG Banking

File: prg_banking.kicad_sch

PPU Address Bus

File: ppu_address_bus.kicad_sch

Reset Detection

File: reset_detection.kicad_sch

CPU Data Bus

File: cpu_data_bus.kicad_sch

PCM DAC

File: pcm_dac.kicad_sch

Scanline Detection

File: scanline_detection.kicad_sch

V-Split

File: v_split.kicad_sch

PPU Fetch Counting

File: ppu_fetch_counting.kicad_sch

CHR A0,A1,A2

File: chr_a0_a1_a2.kicad_sch

ExRAM Mode

File: exram_mode.kicad_sch

Fill Mode

File: fill_mode.kicad_sch

Pulse Length Lookup Table

File: pulse_length_lookup_table.kicad_sch

Pulse Timer

File: pulse_timer.kicad_sch

Pulse Duty Envelope Volume

File: pulse_duty_envelope_volume.kicad_sch

Pulse DAC

File: Pulse DAC.kicad_sch

PPU Data Bus

File: ppu_data_bus.kicad_sch

CHR Bank Registers

File: chr_bank_registers.kicad_sch

CHR Bank Selection Logic

File: chr_bank_selection_logic.kicad_sch

CHR Address Bus 10 to 14

File: chr_address_bus_10_to_14.kicad_sch

CHR Address Bus 15 to 19

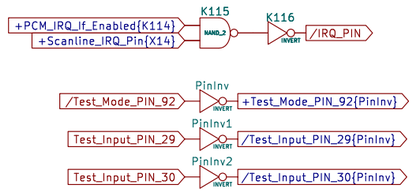
File: chr_address_bus_15_to_19.kicad_sch

Multiplier

File: multiplier.kicad_sch

Artifacts

File: artifacts.kicad_sch



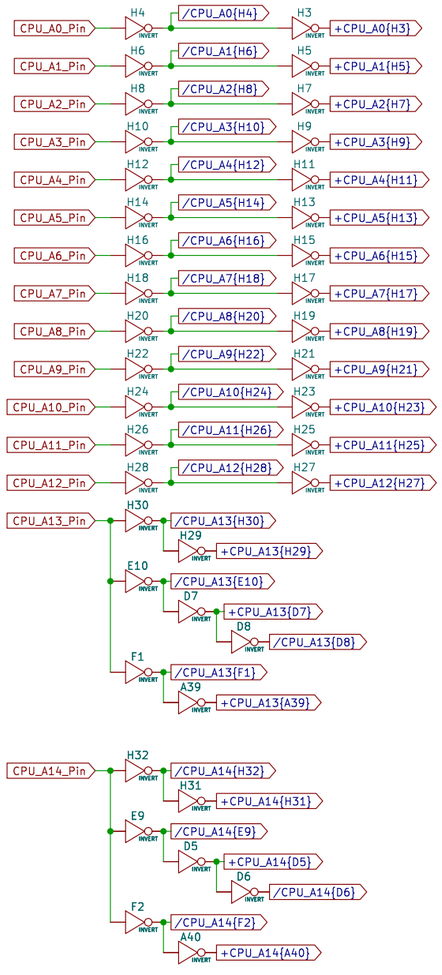
Sheet: /

File: MMC5.kicad_sch

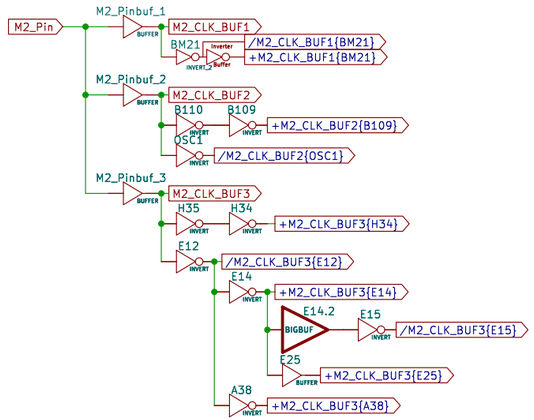
Title:

Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 1/26

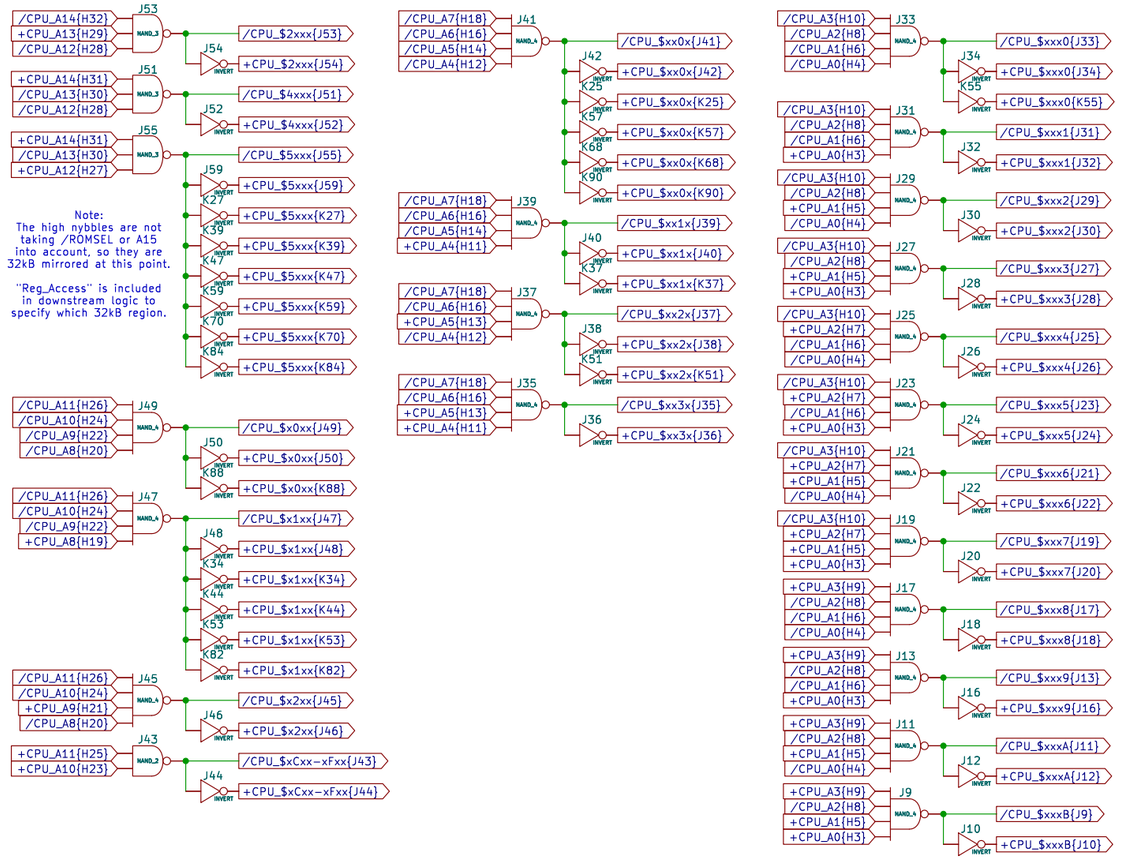
CPU Address Bus Buffering



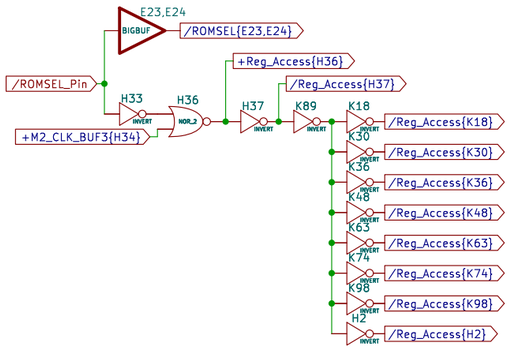
M2 Clock Buffering



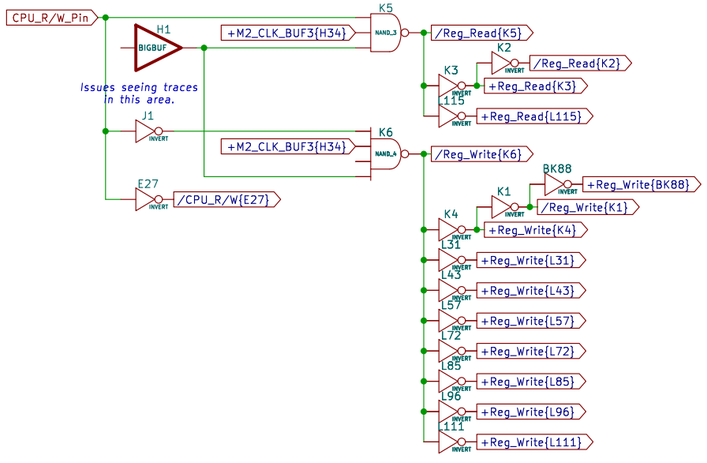
CPU Address Bus Nybble Decoding



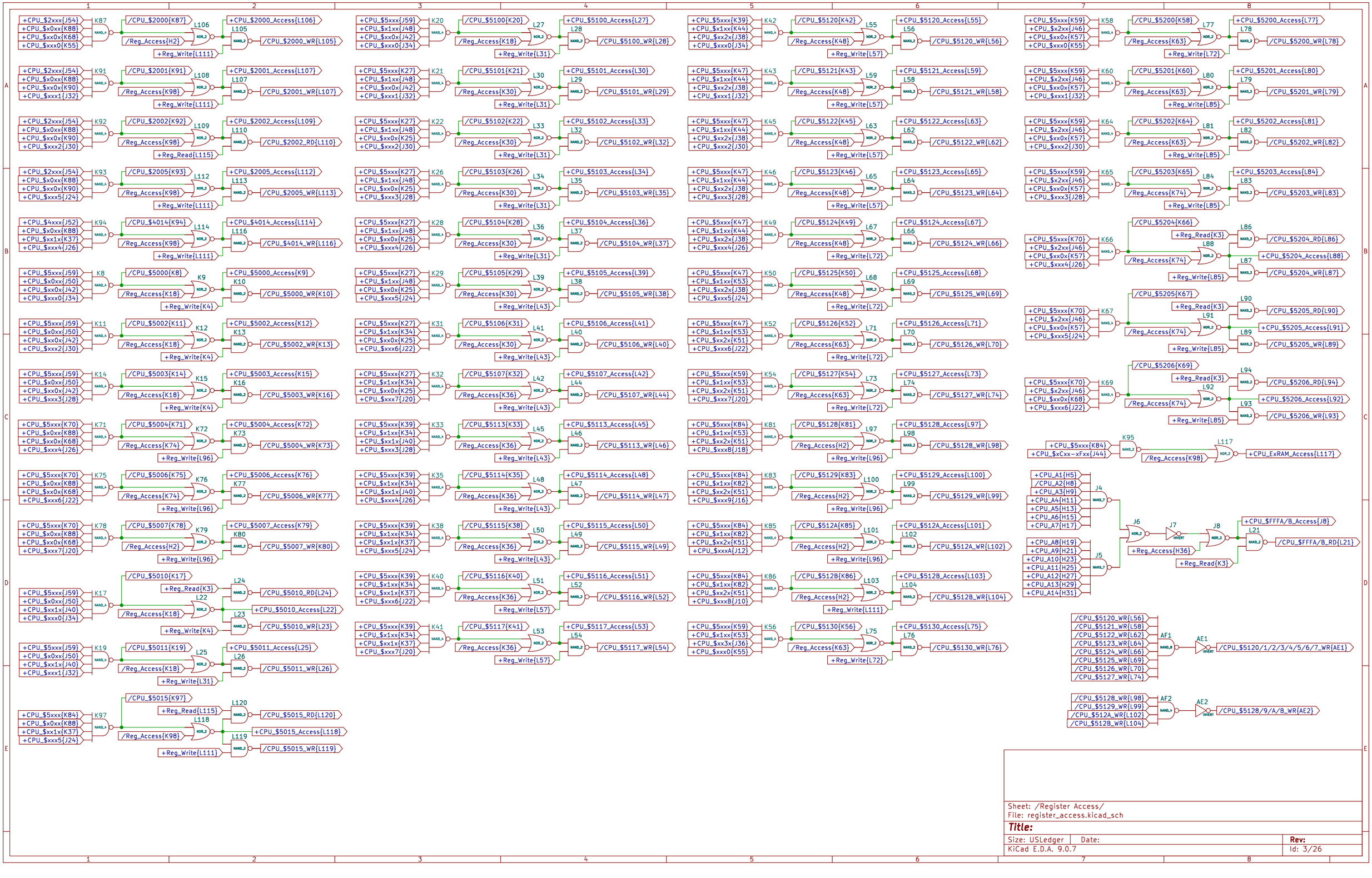
/ROMSEL, Register Access



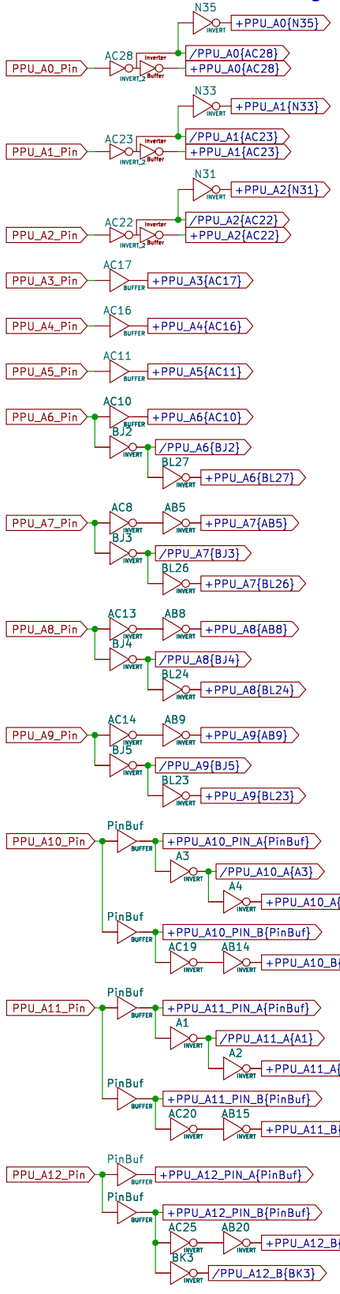
CPU R/W, Register Read/Write

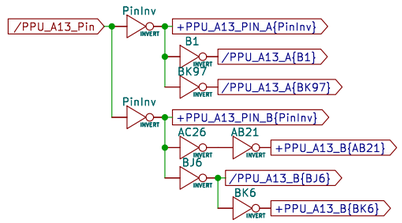


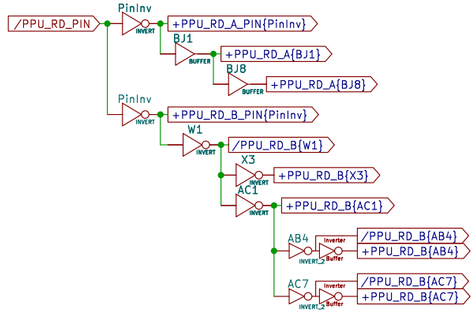
Sheet: /CPU Address Bus/		
File: cpuAddressBus.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 2/26

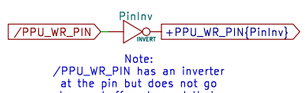

+CPU_$2xxx{J54}
+CPU_$x0xx{K88}
+CPU_$xx0x{K68}
+CPU_$xxx0{K55}
K87
NAND_4
/CPU_$2000{K87}
/Reg_Access{H2}
L106
NOR_2
+CPU_$2000_Access{L106}
L105
NAND_2
/CPU_$2000_WR{L105}
+Reg_Write{L111}
+CPU_$2xxx{J54}
+CPU_$x0xx{K88}
+CPU_$xx0x{K68}
+CPU_$xxx1{J32}
K91
NAND_4
/CPU_$2001{K91}
/Reg_Access{K98}
L108
NOR_2
+CPU_$2001_Access{L107}
L107
NAND_2
/CPU_$2001_WR{L107}
+Reg_Write{L111}
+CPU_$2xxx{J54}
+CPU_$x0xx{K88}
+CPU_$xx0x{K90}
+CPU_$xxx2{J30}
K92
NAND_4
/CPU_$2002{K92}
/Reg_Access{K98}
L109
NOR_2
+CPU_$2002_Access{L109}
L110
NAND_2
/CPU_$2002_RD{L110}
+Reg_Read{L115}
+CPU_$2xxx{J54}
+CPU_$x0xx{K88}
+CPU_$xx0x{K90}
+CPU_$xxx5{J24}
K93
NAND_4
/CPU_$2005{K93}
/Reg_Access{K98}
L112
NOR_2
+CPU_$2005_Access{L112}
L113
NAND_2
/CPU_$2005_WR{L113}
+Reg_Write{L111}
+CPU_$4xxx{J52}
+CPU_$x0xx{K88}
+CPU_$xx1x{K37}
+CPU_$xxx4{J26}
K94
NAND_4
/CPU_$4014{K94}
/Reg_Access{K98}
L114
NOR_2
+CPU_$4014_Access{L114}
L116
NAND_2
/CPU_$4014_WR{L116}
+Reg_Write{L111}
+CPU_$5xxx{J59}
+CPU_$x0xx{J50}
+CPU_$xx0x{J42}
+CPU_$xxx0{J34}
K8
NAND_4
/CPU_$5000{K8}
/Reg_Access{K18}
K9
NOR_2
+CPU_$5000_Access{K9}
K10
NAND_2
/CPU_$5000_WR{K10}
+Reg_Write{K4}
+CPU_$5xxx{J59}
+CPU_$x0xx{J50}
+CPU_$xx0x{J42}
+CPU_$xxx2{J30}
K11
NAND_4
/CPU_$5002{K11}
/Reg_Access{K18}
K12
NOR_2
+CPU_$5002_Access{K12}
K13
NAND_2
/CPU_$5002_WR{K13}
+Reg_Write{K4}
+CPU_$5xxx{J59}
+CPU_$x0xx{J50}
+CPU_$xx0x{J42}
+CPU_$xxx3{J28}
K14
NAND_4
/CPU_$5003{K14}
/Reg_Access{K18}
K15
NOR_2
+CPU_$5003_Access{K15}
K16
NAND_2
/CPU_$5003_WR{K16}
+Reg_Write{K4}
+CPU_$5xxx{K70}
+CPU_$x0xx{K88}
+CPU_$xx0x{K68}
+CPU_$xxx4{J26}
K71
NAND_4
/CPU_$5004{K71}
/Reg_Access{K74}
K72
NOR_2
+CPU_$5004_Access{K72}
K73
NAND_2
/CPU_$5004_WR{K73}
+Reg_Write{L96}
+CPU_$5xxx{K70}
+CPU_$x0xx{K88}
+CPU_$xx0x{K68}
+CPU_$xxx6{J22}
K75
NAND_4
/CPU_$5006{K75}
/Reg_Access{K74}
K76
NOR_2
+CPU_$5006_Access{K76}
K77
NAND_2
/CPU_$5006_WR{K77}
+Reg_Write{L96}
+CPU_$5xxx{K70}
+CPU_$x0xx{K88}
+CPU_$xx0x{K68}
+CPU_$xxx7{J20}
K78
NAND_4
/CPU_$5007{K78}
/Reg_Access{H2}
K79
NOR_2
+CPU_$5007_Access{K79}
K80
NAND_2
/CPU_$5007_WR{K80}
+Reg_Write{L96}
+CPU_$5xxx{J59}
+CPU_$x0xx{J50}
+CPU_$xx1x{J40}
+CPU_$xxx0{J34}
K17
NAND_4
/CPU_$5010{K17}
/Reg_Access{K18}
L22
NOR_2
+Reg_Read{K3}
L24
NAND_2
/CPU_$5010_RD{L24}
+CPU_$5010_Access{L22}
L23
NAND_2
/CPU_$5010_WR{L23}
+Reg_Write{K4}
+CPU_$5xxx{J59}
+CPU_$x0xx{J50}
+CPU_$xx1x{J40}
+CPU_$xxx1{J32}
K19
NAND_4
/CPU_$5011{K19}
/Reg_Access{K18}
L25
NOR_2
+CPU_$5011_Access{L25}
L26
NAND_2
/CPU_$5011_WR{L26}
+Reg_Write{L31}
+CPU_$5xxx{K84}
+CPU_$x0xx{K88}
+CPU_$xx1x{K37}
+CPU_$xxx5{J24}
K97
NAND_4
/CPU_$5015{K97}
/Reg_Access{K98}
L118
NOR_2
+Reg_Read{L115}
L120
NAND_2
/CPU_$5015_RD{L120}
+CPU_$5015_Access{L118}
L119
NAND_2
/CPU_$5015_WR{L119}
+Reg_Write{L111}
+CPU_$5xxx{J59}
+CPU_$x1xx{J48}
+CPU_$xx0x{J42}
+CPU_$xxx0{J34}
K20
NAND_4
/CPU_$5100{K20}
/Reg_Access{K18}
L27
NOR_2
+CPU_$5100_Access{L27}
L28
NAND_2
/CPU_$5100_WR{L28}
+Reg_Write{L31}
+CPU_$5xxx{K27}
+CPU_$x1xx{J48}
+CPU_$xx0x{J42}
+CPU_$xxx1{J32}
K21
NAND_4
/CPU_$5101{K21}
/Reg_Access{K30}
L30
NOR_2
+CPU_$5101_Access{L30}
L29
NAND_2
/CPU_$5101_WR{L29}
+Reg_Write{L31}
+CPU_$5xxx{K27}
+CPU_$x1xx{J48}
+CPU_$xx0x{K25}
+CPU_$xxx2{J30}
K22
NAND_4
/CPU_$5102{K22}
/Reg_Access{K30}
L33
NOR_2
+CPU_$5102_Access{L33}
L32
NAND_2
/CPU_$5102_WR{L32}
+Reg_Write{L31}
+CPU_$5xxx{K27}
+CPU_$x1xx{J48}
+CPU_$xx0x{K25}
+CPU_$xxx3{J28}
K26
NAND_4
/CPU_$5103{K26}
/Reg_Access{K30}
L34
NOR_2
+CPU_$5103_Access{L34}
L35
NAND_2
/CPU_$5103_WR{L35}
+Reg_Write{L31}
+CPU_$5xxx{K27}
+CPU_$x1xx{J48}
+CPU_$xx0x{K25}
+CPU_$xxx4{J26}
K28
NAND_4
/CPU_$5104{K28}
/Reg_Access{K30}
L36
NOR_2
+CPU_$5104_Access{L36}
L37
NAND_2
/CPU_$5104_WR{L37}
+Reg_Write{L31}
+CPU_$5xxx{K27}
+CPU_$x1xx{J48}
+CPU_$xx0x{K25}
+CPU_$xxx5{J24}
K29
NAND_4
/CPU_$5105{K29}
/Reg_Access{K30}
L39
NOR_2
+CPU_$5105_Access{L39}
L38
NAND_2
/CPU_$5105_WR{L38}
+Reg_Write{L43}
+CPU_$5xxx{K27}
+CPU_$x1xx{K34}
+CPU_$xx0x{K25}
+CPU_$xxx6{J22}
K31
NAND_4
/CPU_$5106{K31}
/Reg_Access{K30}
L41
NOR_2
+CPU_$5106_Access{L41}
L40
NAND_2
/CPU_$5106_WR{L40}
+Reg_Write{L43}
+CPU_$5xxx{K27}
+CPU_$x1xx{K34}
+CPU_$xx0x{K25}
+CPU_$xxx7{J20}
K32
NAND_4
/CPU_$5107{K32}
/Reg_Access{K36}
L42
NOR_2
+CPU_$5107_Access{L42}
L44
NAND_2
/CPU_$5107_WR{L44}
+Reg_Write{L43}
+CPU_$5xxx{K39}
+CPU_$x1xx{K34}
+CPU_$xx1x{J40}
+CPU_$xxx3{J28}
K33
NAND_4
/CPU_$5113{K33}
/Reg_Access{K36}
L45
NOR_2
+CPU_$5113_Access{L45}
L46
NAND_2
/CPU_$5113_WR{L46}
+Reg_Write{L43}
+CPU_$5xxx{K39}
+CPU_$x1xx{K34}
+CPU_$xx1x{J40}
+CPU_$xxx4{J26}
K35
NAND_4
/CPU_$5114{K35}
/Reg_Access{K36}
L48
NOR_2
+CPU_$5114_Access{L48}
L47
NAND_2
/CPU_$5114_WR{L47}
+Reg_Write{L43}
+CPU_$5xxx{K39}
+CPU_$x1xx{K34}
+CPU_$xx1x{K37}
+CPU_$xxx5{J24}
K38
NAND_4
/CPU_$5115{K38}
/Reg_Access{K36}
L50
NOR_2
+CPU_$5115_Access{L50}
L49
NAND_2
/CPU_$5115_WR{L49}
+Reg_Write{L43}
+CPU_$5xxx{K39}
+CPU_$x1xx{K34}
+CPU_$xx1x{K37}
+CPU_$xxx6{J22}
K40
NAND_4
/CPU_$5116{K40}
/Reg_Access{K36}
L51
NOR_2
+CPU_$5116_Access{L51}
L52
NAND_2
/CPU_$5116_WR{L52}
+Reg_Write{L57}
+CPU_$5xxx{K39}
+CPU_$x1xx{K34}
+CPU_$xx1x{K37}
+CPU_$xxx7{J20}
K41
NAND_4
/CPU_$5117{K41}
/Reg_Access{K36}
L53
NOR_2
+CPU_$5117_Access{L53}
L54
NAND_2
/CPU_$5117_WR{L54}
+Reg_Write{L57}
+CPU_$5xxx{J39}
+CPU_$x1xx{K44}
+CPU_$xx2x{J38}
+CPU_$xxx0{J34}
K42
NAND_4
/CPU_$5120{K42}
/Reg_Access{K48}
L55
NOR_2
+CPU_$5120_Access{L55}
L56
NAND_2
/CPU_$5120_WR{L56}
+Reg_Write{L57}
+CPU_$5xxx{K47}
+CPU_$x1xx{K44}
+CPU_$xx2x{J38}
+CPU_$xxx1{J32}
K43
NAND_4
/CPU_$5121{K43}
/Reg_Access{K48}
L59
NOR_2
+CPU_$5121_Access{L59}
L58
NAND_2
/CPU_$5121_WR{L58}
+Reg_Write{L57}
+CPU_$5xxx{K47}
+CPU_$x1xx{K44}
+CPU_$xx2x{J38}
+CPU_$xxx2{J30}
K45
NAND_4
/CPU_$5122{K45}
/Reg_Access{K48}
L63
NOR_2
+CPU_$5122_Access{L63}
L62
NAND_2
/CPU_$5122_WR{L62}
+Reg_Write{L57}
+CPU_$5xxx{K47}
+CPU_$x1xx{K44}
+CPU_$xx2x{J38}
+CPU_$xxx3{J28}
K46
NAND_4
/CPU_$5123{K46}
/Reg_Access{K48}
L65
NOR_2
+CPU_$5123_Access{L65}
L64
NAND_2
/CPU_$5123_WR{L64}
+Reg_Write{L57}
+CPU_$5xxx{K47}
+CPU_$x1xx{K44}
+CPU_$xx2x{J38}
+CPU_$xxx4{J26}
K49
NAND_4
/CPU_$5124{K49}
/Reg_Access{K48}
L67
NOR_2
+CPU_$5124_Access{L67}
L66
NAND_2
/CPU_$5124_WR{L66}
+Reg_Write{L72}
+CPU_$5xxx{K47}
+CPU_$x1xx{K53}
+CPU_$xx2x{J38}
+CPU_$xxx5{J24}
K50
NAND_4
/CPU_$5125{K50}
/Reg_Access{K48}
L68
NOR_2
+CPU_$5125_Access{L68}
L69
NAND_2
/CPU_$5125_WR{L69}
+Reg_Write{L72}
+CPU_$5xxx{K47}
+CPU_$x1xx{K53}
+CPU_$xx2x{K51}
+CPU_$xxx6{J22}
K52
NAND_4
/CPU_$5126{K52}
/Reg_Access{K63}
L71
NOR_2
+CPU_$5126_Access{L71}
L70
NAND_2
/CPU_$5126_WR{L70}
+Reg_Write{L72}
+CPU_$5xxx{K59}
+CPU_$x1xx{K53}
+CPU_$xx2x{K51}
+CPU_$xxx7{J20}
K54
NAND_4
/CPU_$5127{K54}
/Reg_Access{K63}
L73
NOR_2
+CPU_$5127_Access{L73}
L74
NAND_2
/CPU_$5127_WR{L74}
+Reg_Write{L72}
+CPU_$5xxx{K84}
+CPU_$x1xx{K53}
+CPU_$xx2x{K51}
+CPU_$xxx8{J18}
K81
NAND_4
/CPU_$5128{K81}
/Reg_Access{H2}
L97
NOR_2
+CPU_$5128_Access{L97}
L98
NAND_2
/CPU_$5128_WR{L98}
+Reg_Write{L96}
+CPU_$5xxx{K84}
+CPU_$x1xx{K82}
+CPU_$xx2x{K51}
+CPU_$xxx9{J16}
K83
NAND_4
/CPU_$5129{K83}
/Reg_Access{H2}
L100
NOR_2
+CPU_$5129_Access{L100}
L99
NAND_2
/CPU_$5129_WR{L99}
+Reg_Write{L96}
+CPU_$5xxx{K84}
+CPU_$x1xx{K82}
+CPU_$xx2x{K51}
+CPU_$xxxA{J12}
K85
NAND_4
/CPU_$512A{K85}
/Reg_Access{H2}
L101
NOR_2
+CPU_$512A_Access{L101}
L102
NAND_2
/CPU_$512A_WR{L102}
+Reg_Write{L96}
+CPU_$5xxx{K84}
+CPU_$x1xx{K82}
+CPU_$xx2x{K51}
+CPU_$xxxB{J10}
K86
NAND_4
/CPU_$512B{K86}
/Reg_Access{H2}
L103
NOR_2
+CPU_$512B_Access{L103}
L104
NAND_2
/CPU_$512B_WR{L104}
+Reg_Write{L111}
+CPU_$5xxx{K59}
+CPU_$x1xx{K53}
+CPU_$xx3x{J36}
+CPU_$xxx0{K55}
K56
NAND_4
/CPU_$5130{K56}
/Reg_Access{K63}
L75
NOR_2
+CPU_$5130_Access{L75}
L76
NAND_2
/CPU_$5130_WR{L76}
+Reg_Write{L72}
+CPU_$5xxx{K59}
+CPU_$x1xx{K46}
+CPU_$xx0x{J34}
+CPU_$xxx0{K55}
K58
NAND_4
/CPU_$5200{K58}
/Reg_Access{K63}
L77
NOR_2
+CPU_$5200_Access{L77}
L78
NAND_2
/CPU_$5200_WR{L78}
+Reg_Write{L72}
+CPU_$5xxx{K59}
+CPU_$x2xx{J46}
+CPU_$xx0x{K57}
+CPU_$xxx1{J32}
K60
NAND_4
/CPU_$5201{K60}
/Reg_Access{K63}
L80
NOR_2
+CPU_$5201_Access{L80}
L79
NAND_2
/CPU_$5201_WR{L79}
+Reg_Write{L85}
+CPU_$5xxx{K59}
+CPU_$x2xx{J46}
+CPU_$xx0x{K57}
+CPU_$xxx2{J30}
K64
NAND_4
/CPU_$5202{K64}
/Reg_Access{K63}
L81
NOR_2
+CPU_$5202_Access{L81}
L82
NAND_2
/CPU_$5202_WR{L82}
+Reg_Write{L85}
+CPU_$5xxx{K59}
+CPU_$x2xx{J46}
+CPU_$xx0x{K57}
+CPU_$xxx3{J28}
K65
NAND_4
/CPU_$5203{K65}
/Reg_Access{K74}
L84
NOR_2
+CPU_$5203_Access{L84}
L83
NAND_2
/CPU_$5203_WR{L83}
+Reg_Write{L85}
+CPU_$5xxx{K70}
+CPU_$x2xx{J46}
+CPU_$xx0x{K57}
+CPU_$xxx4{J26}
K66
NAND_4
/CPU_$5204{K66}
+Reg_Read{K3}
L86
NAND_2
/CPU_$5204_RD{L86}
/Reg_Access{K74}
L88
NOR_2
+CPU_$5204_Access{L88}
L87
NAND_2
/CPU_$5204_WR{L87}
+Reg_Write{L85}
+CPU_$5xxx{K70}
+CPU_$x2xx{J46}
+CPU_$xx0x{K57}
+CPU_$xxx5{J24}
K67
NAND_4
/CPU_$5205{K67}
+Reg_Read{K3}
L90
NAND_2
/CPU_$5205_RD{L90}
/Reg_Access{K74}
L91
NOR_2
+CPU_$5205_Access{L91}
L89
NAND_2
/CPU_$5205_WR{L89}
+Reg_Write{L85}
+CPU_$5xxx{K70}
+CPU_$x2xx{J46}
+CPU_$xx0x{K68}
+CPU_$xxx6{J22}
K69
NAND_4
/CPU_$5206{K69}
+Reg_Read{K3}
L94
NAND_2
/CPU_$5206_RD{L94}
/Reg_Access{K74}
L92
NOR_2
+CPU_$5206_Access{L92}
L93
NAND_2
/CPU_$5206_WR{L93}
+Reg_Write{L85}
+CPU_$5xxx{K84}
+CPU_$xCxx-xFxx{J44}
K95
NAND_2
/Reg_Access{K98}
L117
NOR_2
+CPU_ExRAM_Access{L117}
+CPU_A1{H5}
/CPU_A2{H8}
+CPU_A3{H9}
+CPU_A4{H11}
+CPU_A5{H13}
+CPU_A6{H15}
+CPU_A7{H17}
J4
NAND_7
+CPU_A8{H19}
+CPU_A9{H21}
+CPU_A10{H23}
+CPU_A11{H25}
+CPU_A12{H27}
+CPU_A13{H29}
+CPU_A14{H31}
J5
NAND_7
J6
NOR_2
J7
INVERT
+Reg_Access{H36}
J8
NOR_2
+CPU_$FFFA/B_Access{J8}
L21
NAND_2
/CPU_$FFFA/B_RD{L21}
+Reg_Read{K3}
/CPU_$5120_WR{L56}
/CPU_$5121_WR{L58}
/CPU_$5122_WR{L62}
/CPU_$5123_WR{L64}
/CPU_$5124_WR{L66}
/CPU_$5125_WR{L69}
/CPU_$5126_WR{L70}
/CPU_$5127_WR{L74}
AF1
NAND_8
AE1
INVERT
/CPU_$5120/1/2/3/4/5/6/7_WR{AE1}
/CPU_$5128_WR{L98}
/CPU_$5129_WR{L99}
/CPU_$512A_WR{L102}
/CPU_$512B_WR{L104}
AF2
NAND_4
AE2
INVERT
/CPU_$5128/9/A/B_WR{AE2}
Sheet: /Register Access/
File: register_access.kicad_sch
Title:
Size: USLedger
Date:
Rev:
KiCad E.D.A. 9.0.7
Id: 3/26

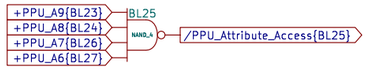
PPU Address Bus Buffering





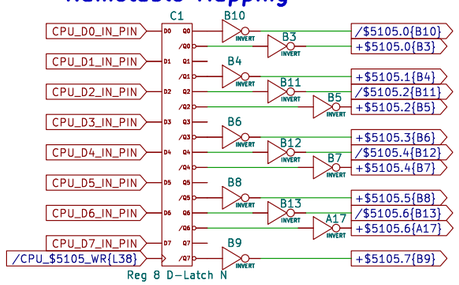


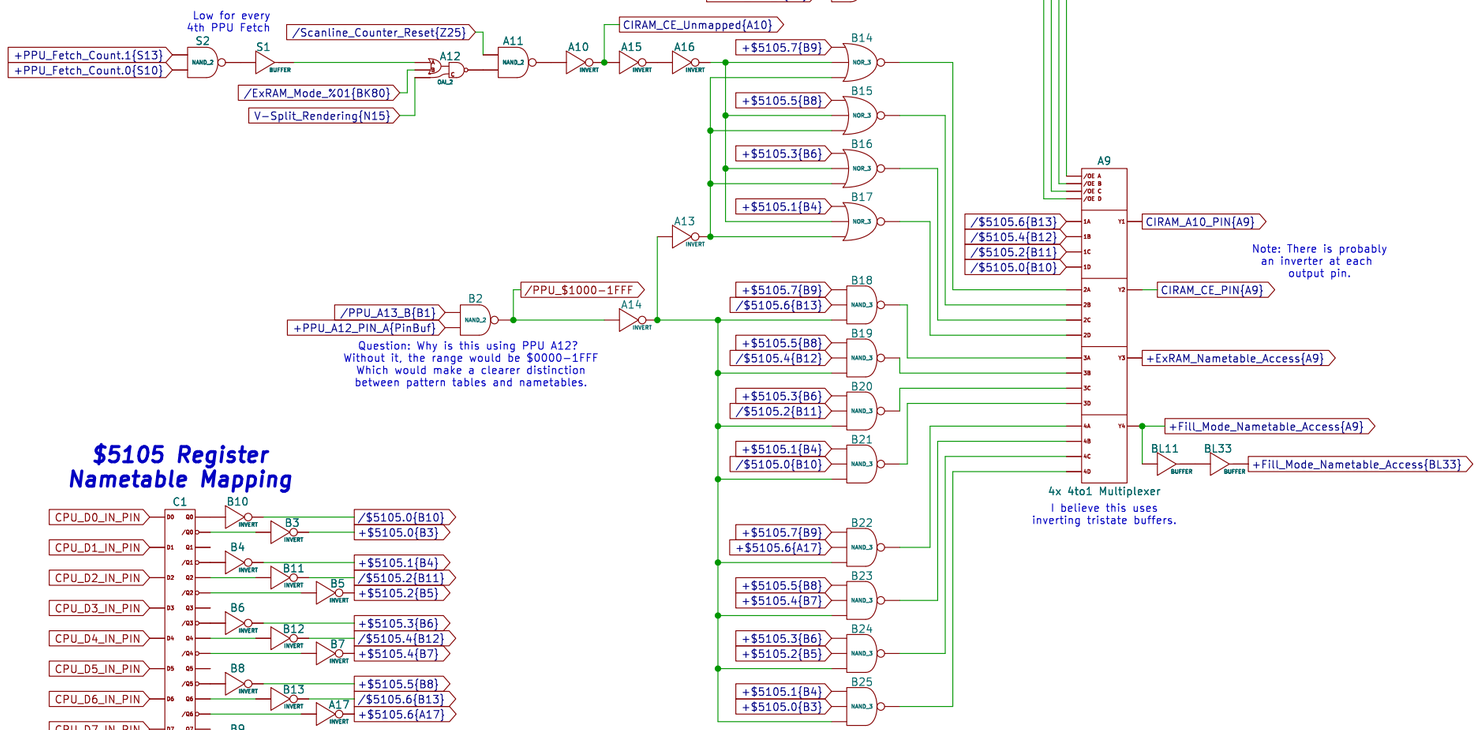




Sheet: /PPU Address Bus/		
File: ppu_address_bus.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 4/26

$5105 Register Nametable Mapping



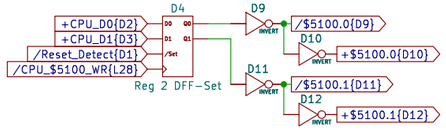


Sheet: /Nametable Mapping/
File: nametable_mapping.kicad_sch

Title:

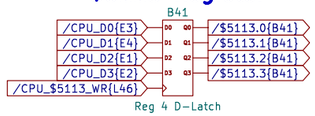
Size: USLedger | Date: | **Rev:**
KiCad E.D.A. 9.0.7 | Id: 5/26

$5100 Register
PRG Mode

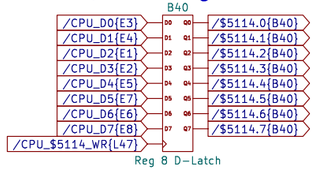


PRG Bank Registers

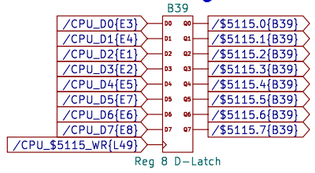
$5113 Register



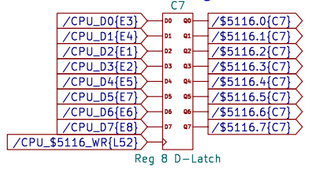
$5114 Register



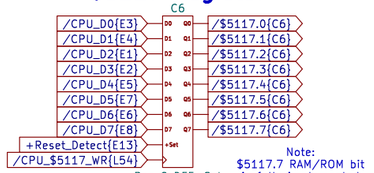
$5115 Register



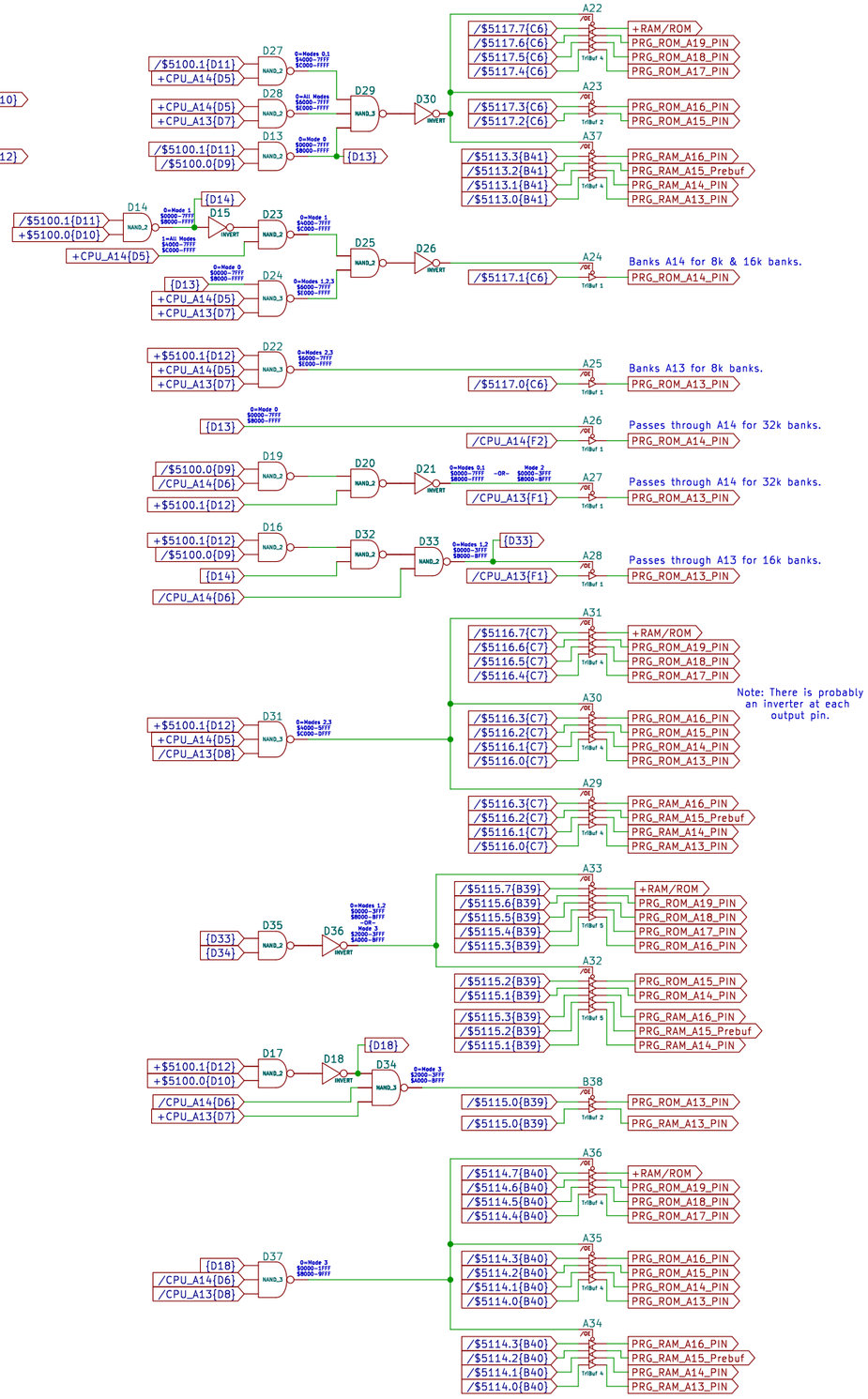
$5116 Register

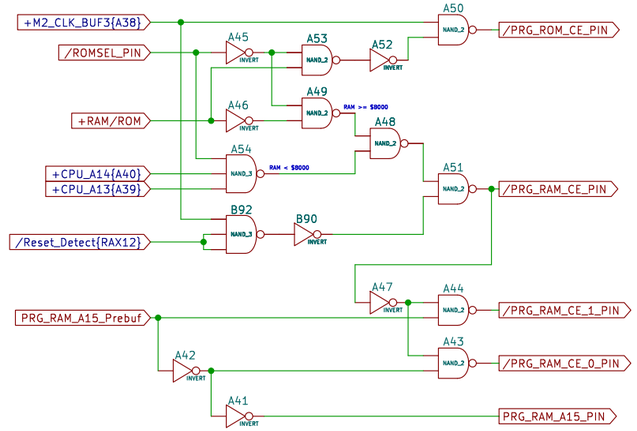


$5117 Register

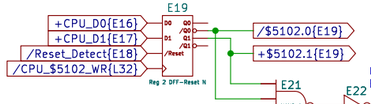


Note: $5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.



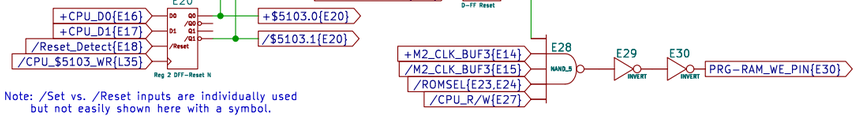


$5102 Register
PRG-RAM Write Protection



$5103 Register

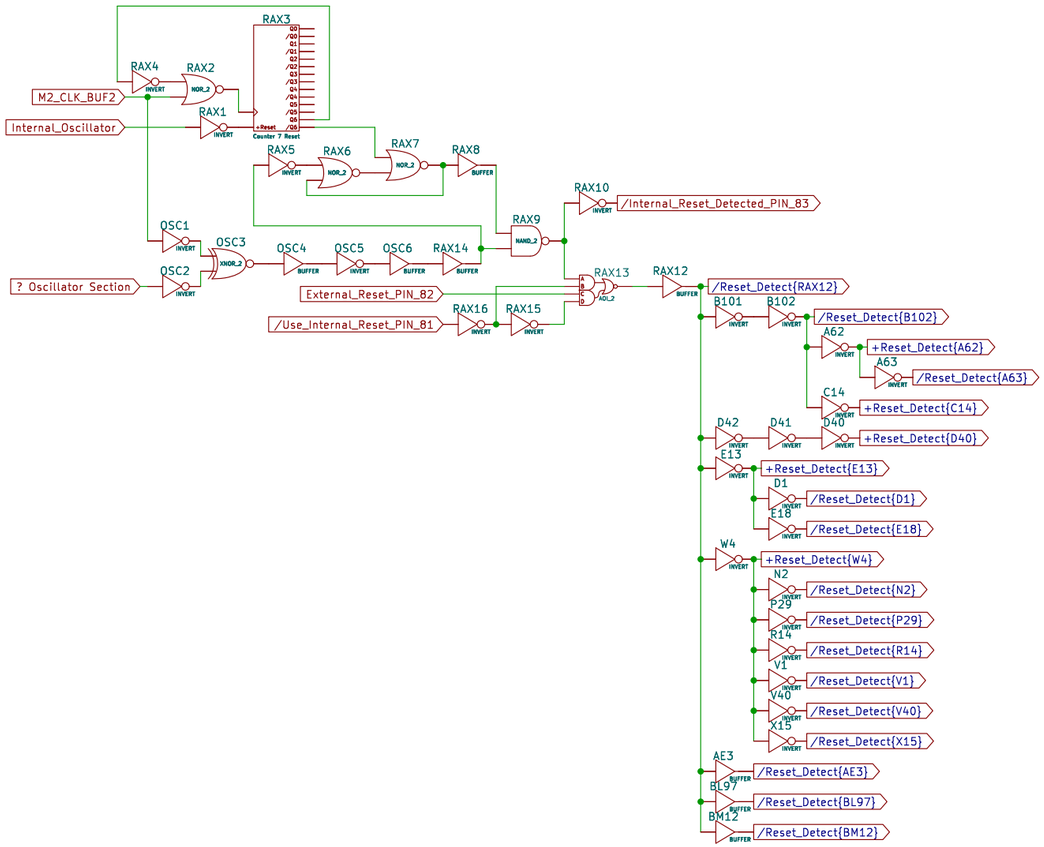
Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

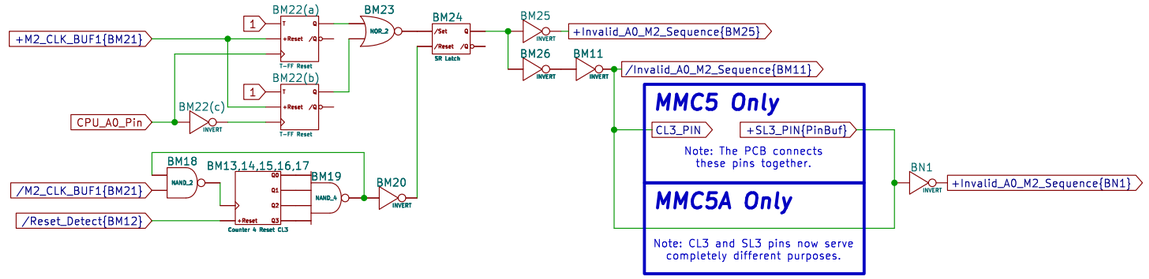


Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

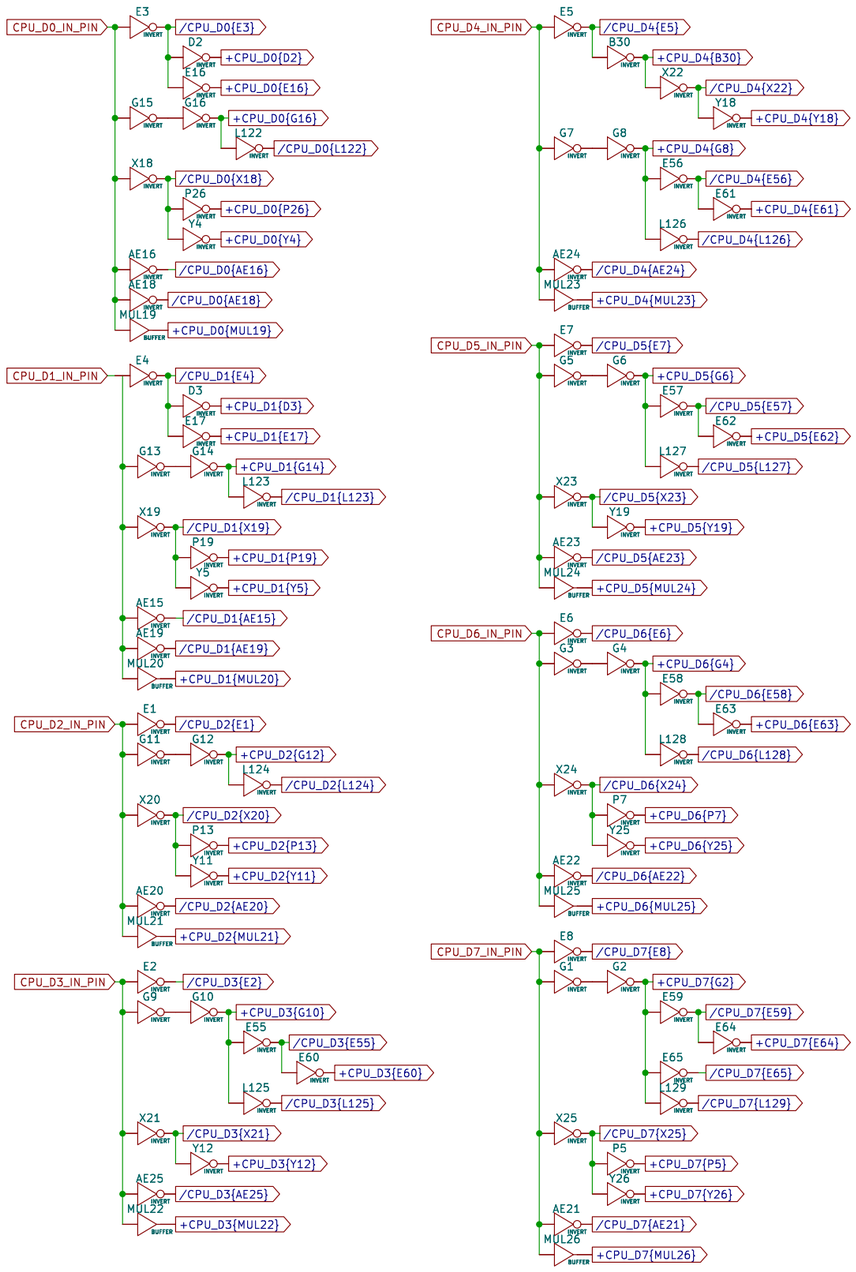
Size: USLedger | Date: | **Rev:**
KiCad E.D.A. 9.0.7 | Id: 6/26



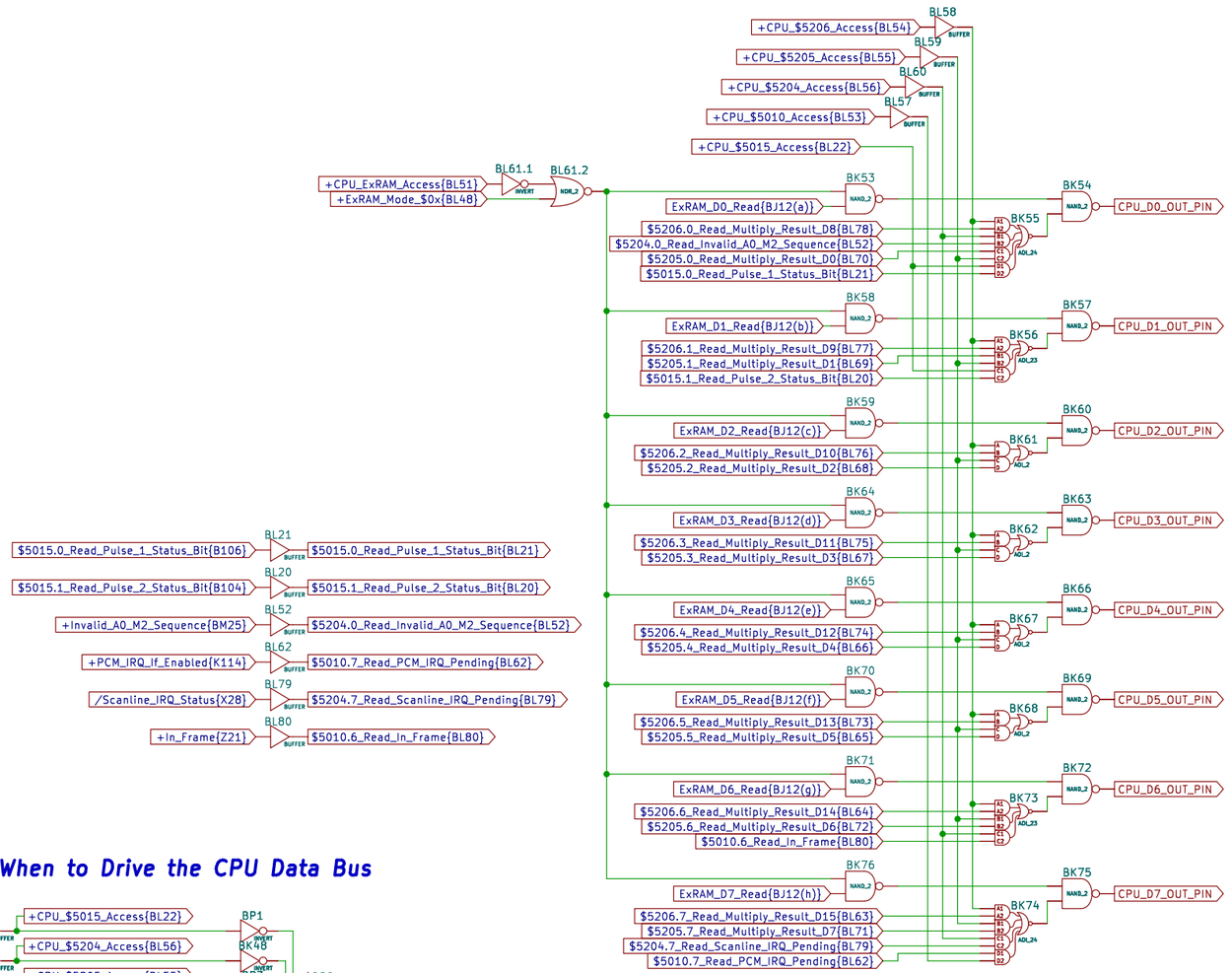


Sheet: /Reset Detection/	
File: reset_detection.kicad_sch	
Title:	
Size: USLedger Date:	**Rev:**
KiCad E.D.A. 9.0.7	Id: 7/26

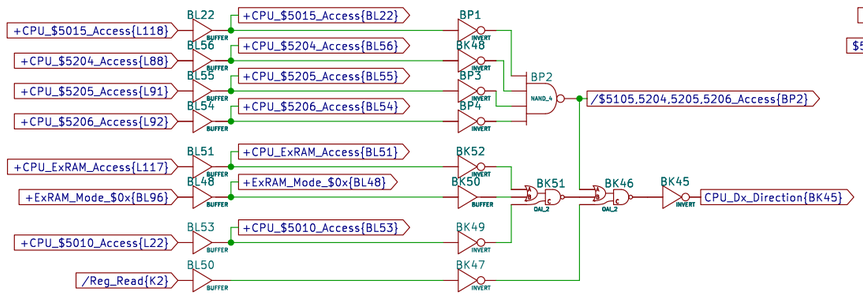
Incoming CPU Data Bus Buffering Bonanza



Multiplexing Data to be Driven onto the CPU Data Bus

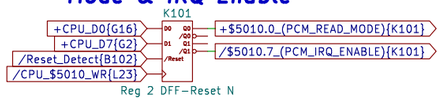


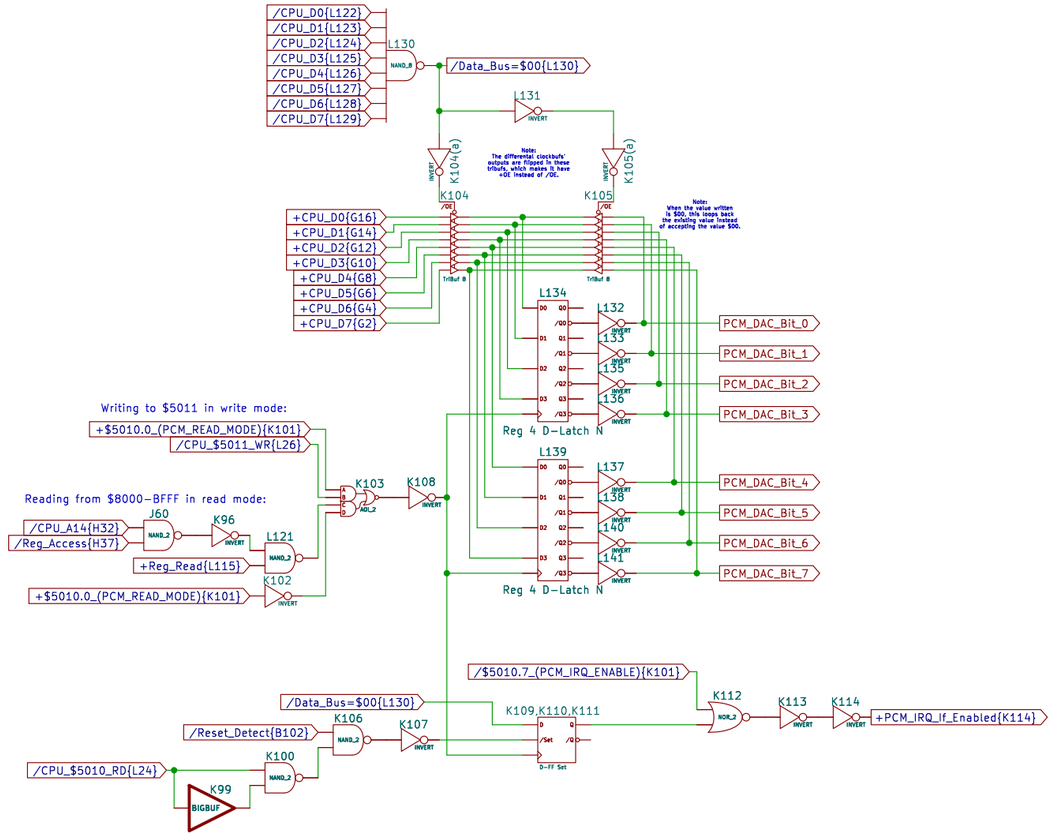
Decides When to Drive the CPU Data Bus



Sheet: /CPU Data Bus/		
File: cpu_data_bus.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 8/26

$5010 Register
Mode & IRQ Enable





Sheet: /PCM DAC/	
File: pcm_dac.kicad_sch	
Title:	
Size: USLedger | Date:	**Rev:**
KiCad E.D.A. 9.0.7	Id: 9/26

Scanline Start Detection

+Use Internal Scanline Detection

+Use External Scanline Detection

+Test_Mode_PIN_92{Pin1nv}

/Test_Input_PIN_29{Pin1nv}

External Scanline Start

High when the present PPU address is from $2xxx and matches the previous read.

/PPU_RD_B{W1}

Low when the 3 most recent PPU reads were from $2xxx and all matched.

/Scanline_Start_Detected{AA3}

/Scanline_Start_Detected{R2}

M2_CLK_BUF1

+PPU_RD_B{AC1}

/Exit_In_Frame{AC6}

Scanline End Detection

Increment_Scanline_Fetch_Count{S6}

+PPU_Cycle_336{R4}

Scanline_End_Detected_Internal{R8}

Scanline_End_Detected_Internal{R8}

/Test_Input_PIN_30{Pin1nv}

External Scanline End

Scanline_End_Detected{P3}

Scanline_End_Detected{T5}

$5204 Register
Scanline IRQ Enable

+CPU_D7{Y26}

/Reset_Detect{X15}

/CPU_$5204_WR{L87}

+Scanline_IRQ_Pin{X14}

$5203 Register
IRQ Scanline
Compare Value

Y2,7,9,14,16,21,23,28

+CPU_D0{Y4}

+CPU_D1{Y5}

+CPU_D2{Y11}

+CPU_D3{Y12}

+CPU_D4{Y18}

+CPU_D5{Y19}

+CPU_D6{Y25}

+CPU_D7{Y26}

/CPU_$5203_WR{L83}

Reg 8 D-Latch

Hi when $5203 matches the present V-Split scanline.

D'Morgan tells us that we can pretend Y8,Y15,Y22 are all AND gates.

IRQ Scanline
Counter

/Scanline_Start_Detected{AA3}

+In_Frame{Z21}

/In_Frame{Z23}

/Scanline_240_and_up{Z22}

$2001 Register
Write Snooping

+CPU_D3{Y12}

+CPU_D4{Y18}

/CPU_$2001_WR{L107}

Hi = PPU Rendering Enabled

/CPU_$5204_RD{L86}

+New_Scanline_Detected{AA3 via Z23}

/In_Frame{Z23}

/Scanline_Start_Detected{AA3}

+IRQ

/IRQ

/Scanline_IRQ_Status{X28}

/CPU_$5204_RD{L86}

$2002 Register
Read Snooping

Latches $2002.7 value read by CPU for 1 CPU cycle.

/CPU_$2002_RD{L110}

+CPU_D7{Y26}

M2_CLK_BUF1

/CPU_$FFFA/B_RD{L21}

/CPU_$4014_WR{L116}

/Exit_In_Frame{AC6}

/Scanline_Start_Detected{AA3}

/In_Frame{Z23}

+In_Frame{Z21}

+Rendering_In_Frame{Z25}

/Rendering_In_Frame{V39}

/Rendering_In_Frame{R24}

+Rendering_In_Frame{N9}

+PPU_A13_B{AB21}

+PPU_A12_B{AB20}

+PPU_A11_B{AB15}

+PPU_A10_B{AB14}

+PPU_A9{AB9}

+PPU_A8{AB8}

+PPU_A7{AB5}

+PPU_A6{AC10}

+PPU_A5{AC11}

+PPU_A4{AC16}

+PPU_A3{AC17}

+PPU_A2{AC22}

+PPU_A1{AC23}

+PPU_A0{AC28}

+PPU_RD_B{AB4}

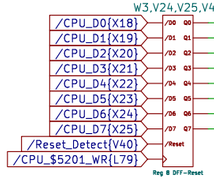
/Exit_In_Frame{AC6}

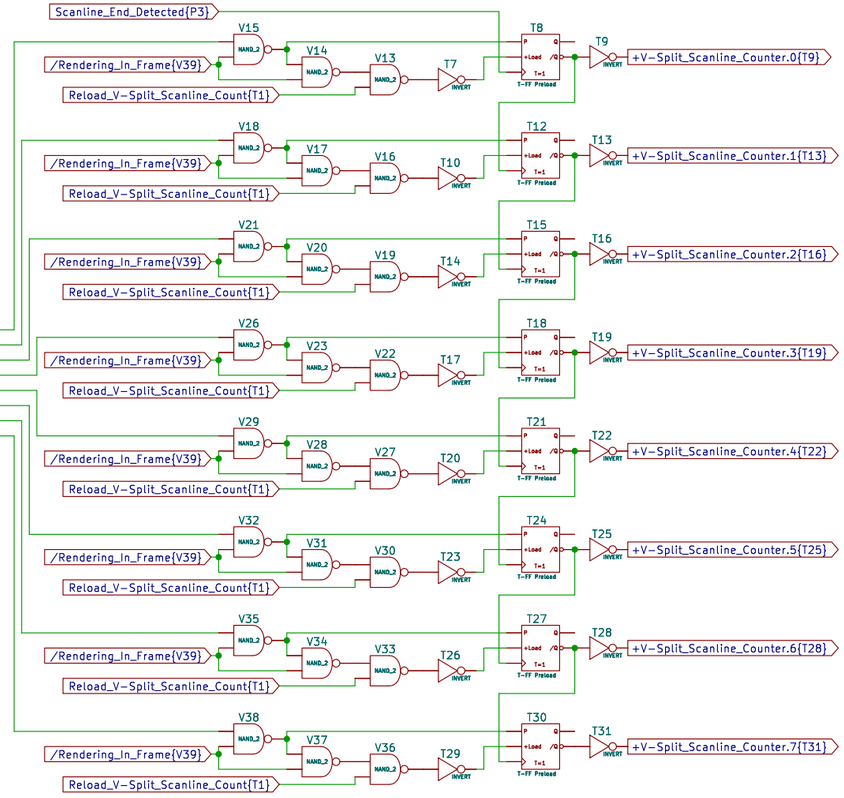
+PPU_RD_B{AC7}

When not in frame, the previous PPU read address always looks like $0000. Since A13, A12 are looking at range $2xxx instead of matches, $0xxx won't match $2xxx. Therefore, scanlines don't get detected when using PPUDATA to read from $0000 during v-blank.

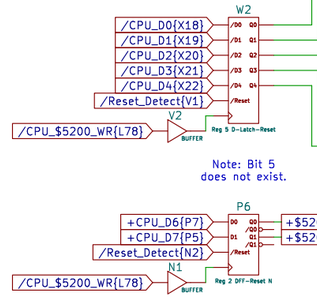
V−Split Scanline Counter

$5201 Register V−Split V−Scroll

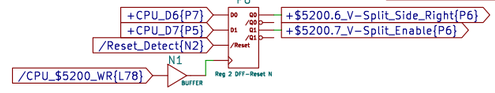


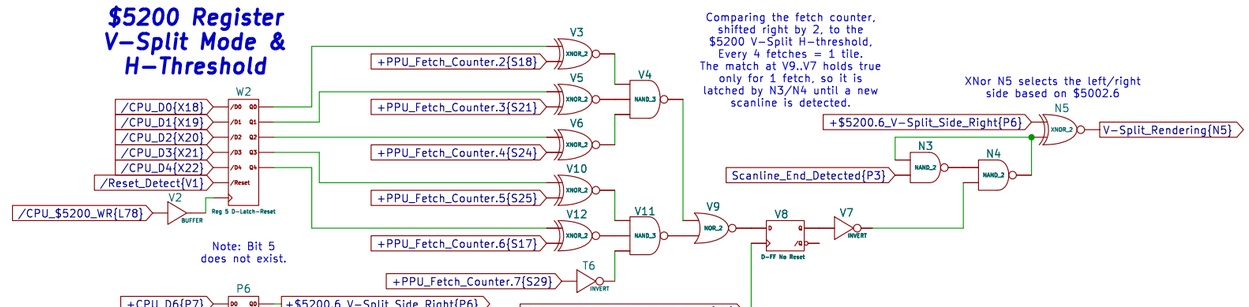


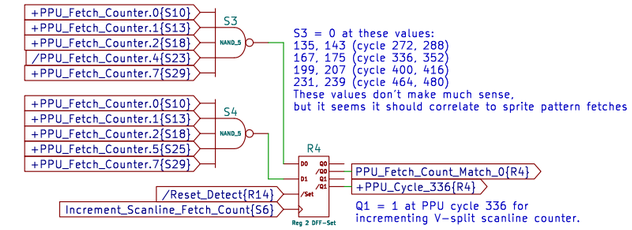
$5200 Register V−Split Mode & H−Threshold



Note: Bit 5 does not exist.



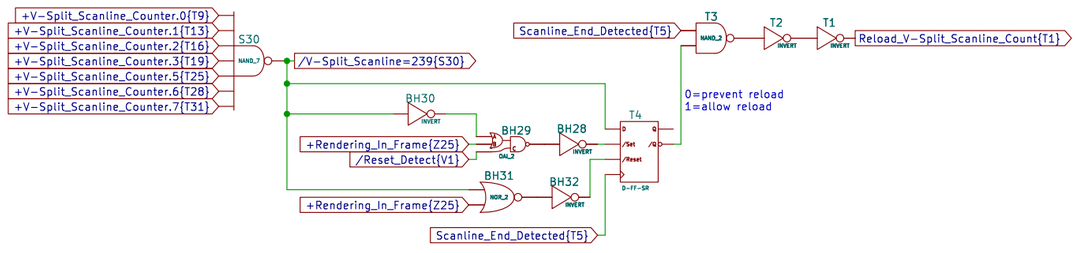




S3 = 0 at these values:
135, 143 (cycle 272, 288)
167, 175 (cycle 336, 352)
199, 207 (cycle 400, 416)
231, 239 (cycle 464, 480)
These values don't make much sense, but it seems it should correlate to sprite pattern fetches

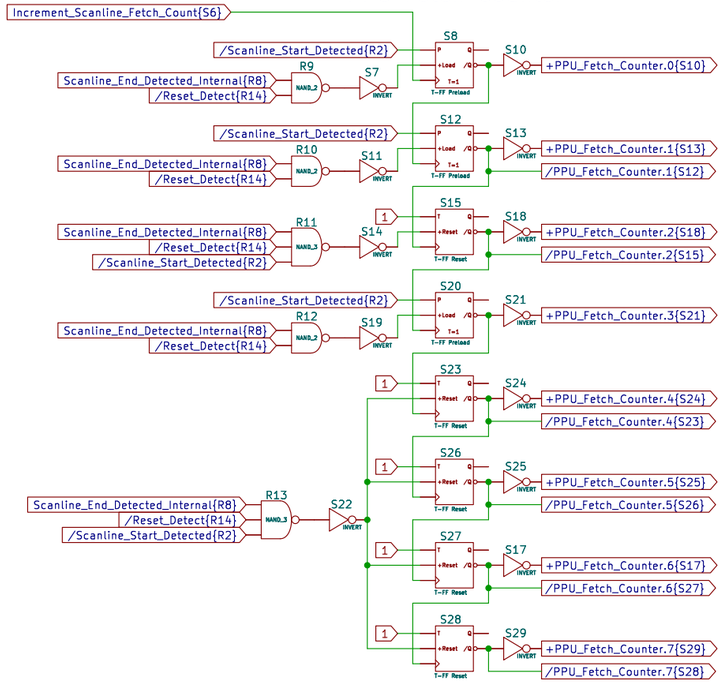
Q1 = 1 at PPU cycle 336 for incrementing V−split scanline counter.

Logic from S4:
Value 1x1xx111, when incrementing,
First triggered at 10100111=0xA7=167.
Because fetch count clocks the latch,
167+1 = 168. Since PPU has 2 cycles per fetch, 168*2=336 cycles.
This aligns with "unused NT fetches" at the end of the scanline.

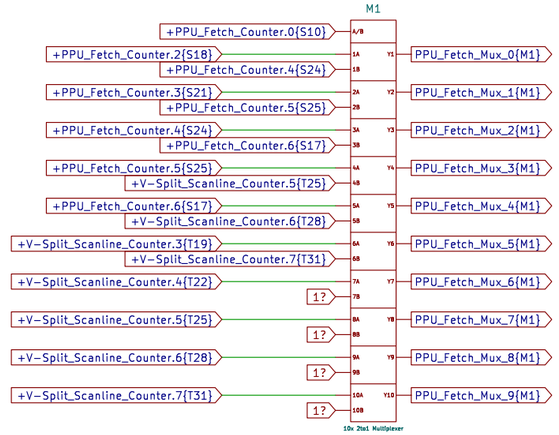


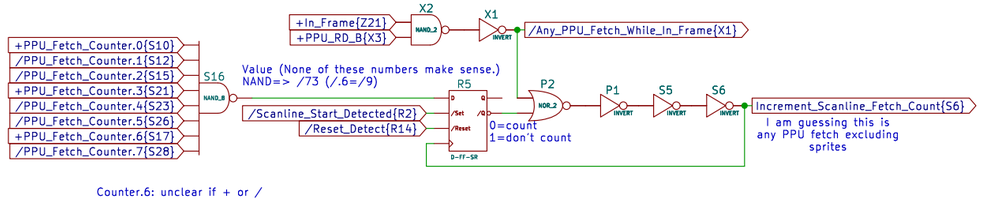
Sheet: /V−Split/		
File: v_split.kicad_sch		
Title:		
Size: B	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 11/26

PPU Fetch Counter



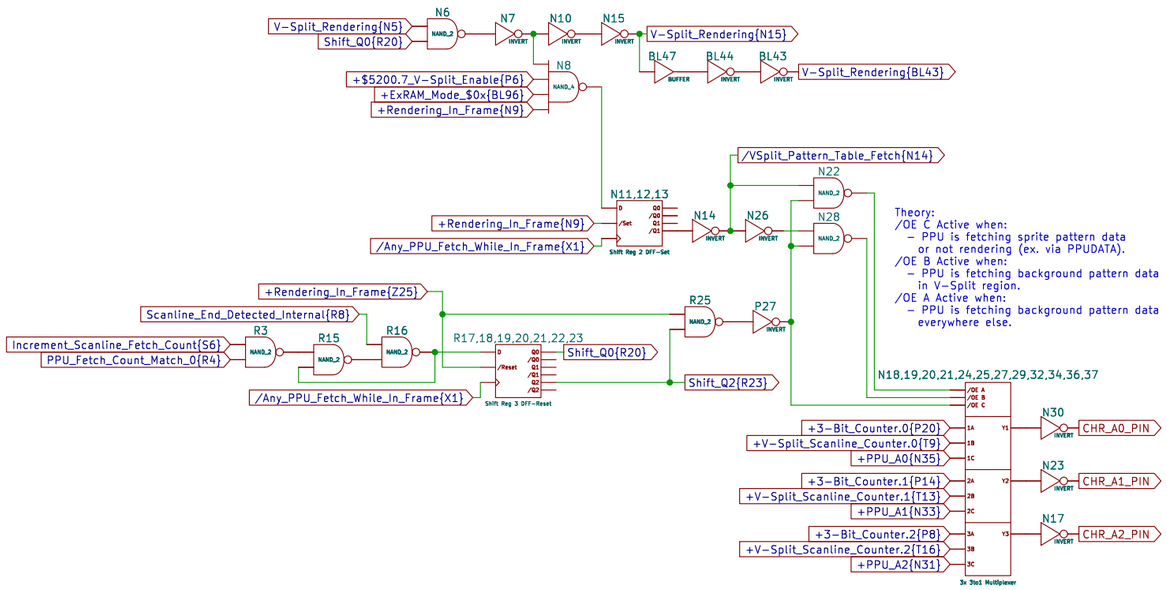
ExRAM Attribute Fetch Mux

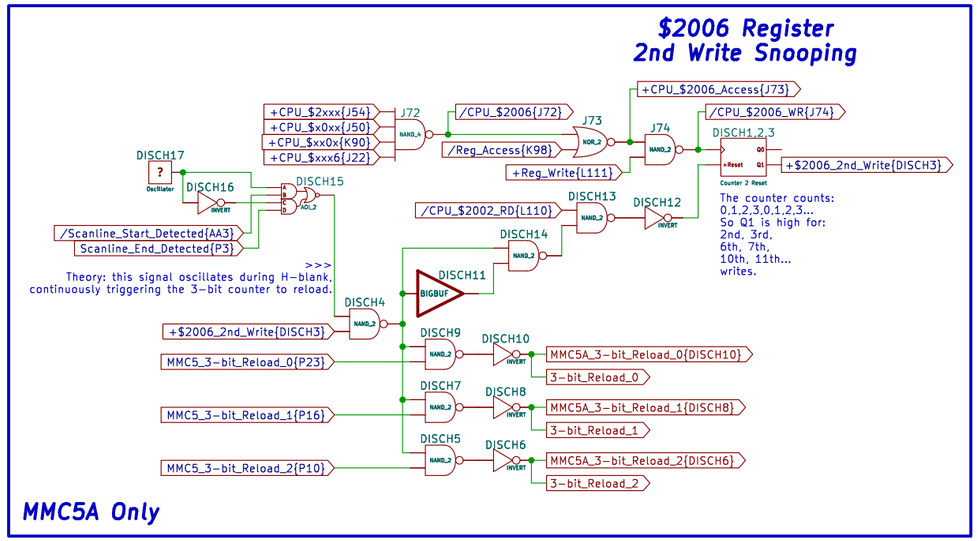


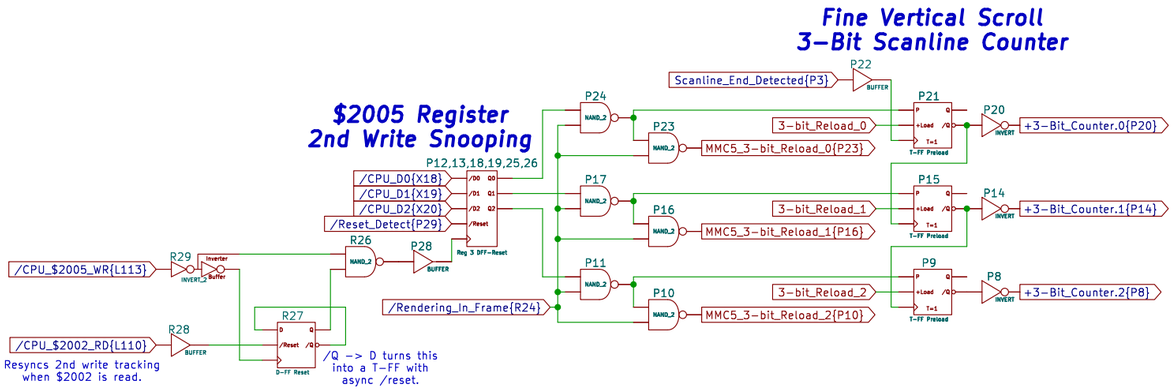


Counter.6: unclear if + or /

Sheet: /PPU Fetch Counting/		
File: ppu_fetch_counting.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 12/26







Sheet: /CHR A0,A1,A2/		
File: chr_a0_a1_a2.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 13/26

$5104 Register
ExRAM Mode

CPU_D0_IN_PIN
CPU_D1_IN_PIN
/Reset_Detect{BL97}
/CPU_$5104_WR{L37}
BK82
Reg 2 DFF-Set
BK79
BK87
+ExRAM_Mode_%10{BK87}
BK80
BL49
/ExRAM_Mode_%01{BK80}
/ExRAM_Mode_%01{BL49}
BK81
BL96
/ExRAM_Mode_%00{BK81}
Appears to go nowhere.
+ExRAM_Mode_$0x{BL96}
Tracks whether $5104 has been written since reset detection
/Reset_Detect{RAX12}
M2_CLK_BUF1
+Invalid_A0_M2_Sequence{BN1}
BL89
BL90
BL91
SR Latch
Syncs SL3 to M2
BL98
D-FF Reset
BL94
BL92
BL93
BN2
BIGBUF
There appears to be a selectable delay jumper, connected at BM6.
BM10 BM9 BM8 BM7 BM6 BM5 BM4 BM3 BM2 BM1
BUFFER
+ExRAM_Mode_$0x{BL96}
BL86
BL88
BL85
BL84
BL83
PPU_RW_Mode_%0x_Access{BL83}
Not sure about polarity; this may be Mode %1x Access, and active low or high.
+PPU_RD_A_PIN{PinInv}
+PPU_WR_PIN{PinInv}
BL82
BL87
BL81
BIGBUF

+CPU_ExRAM_Access{L117}
+Reg_Write{BK88}
BK85
/CPU_Write_To_ExRAM_Any_Mode{BK85}
BK86
+ExRAM_Mode_%10{BK87}
Writing to ExRAM via $5C00-5FFF in mode %10
Writing to an ExRAM nametable via PPU via $2000
+ExRAM_Nametable_Access{A9}
+ExRAM_Mode_$0x{BL96}
+PPU_WR_PIN{PinInv}
BK83
BK84
+ExRAM_WR{BK84}
This is the write signal for the ExRAM itself.

BN4
BIGBUF
BL105
BL106
BL109
/Q{BL109}
+ExRAM_Mode_$0x{BL96}
/CPU_Write_To_ExRAM_Any_Mode{BK85}
BL108
BL99
/Q{BL99}
/PPU_A13_A{BK97}
BL102
BL101
BL100
/Reset_Detect{RAX12}
BN3
BIGBUF
BL107
+PPU_RD_A_PIN{PinInv}

/PPU_A13_A{BK97}
+ExRAM_Mode_$0x{BL96}
BK94
BK95
+Drive_ExRAM_Ax_From_PPU_Ax{BK95}
V-Split_Rendering{N15}
BK92
BK93
BK91
+Drive_ExRAM_Ax_For_Substituted_Fetch{BK91}
BK96
BJ51
+Drive_ExRAM_Ax_From_CPU_Ax{BJ51}
TRUE when:
ExRAM is mode $1x, which gives CPU access but no PPU access, *OR* PPU is accessing NT or AT
That doesn't make sense because what if you have a nametable assigned to ExRAM in $5105? It needs ExRAM Ax coming from PPU.
Signal A9 (+ExRAM Nametable Access) is VERY closeby but no obvious connection
Consider:
ExRAM NEVER contains pattern tables and that should be related to how PPU /A13 is being used here.
Question:
Why should we care about PPU /A13 AT ALL when the CPU has control of ExRAM address bus?
Consider:
ExRAM can still be used as a nametable when ExRAM mode is set to CPU mode.

BJ50 PPU_Fetch_Mux_0{M1} PPU_A0_Pin ExRAM_A0{BJ50}
BJ49 PPU_Fetch_Mux_1{M1} PPU_A1_Pin ExRAM_A1{BJ49}
BJ48 PPU_Fetch_Mux_2{M1} PPU_A2_Pin ExRAM_A2{BJ48}
BJ47 PPU_Fetch_Mux_3{M1} PPU_A3_Pin ExRAM_A3{BJ47}
BJ46 PPU_Fetch_Mux_4{M1} PPU_A4_Pin ExRAM_A4{BJ46}
BJ45 PPU_Fetch_Mux_5{M1} PPU_A5_Pin ExRAM_A5{BJ45}
BJ44 PPU_Fetch_Mux_6{M1} PPU_A6_Pin ExRAM_A6{BJ44}
BJ43 PPU_Fetch_Mux_7{M1} PPU_A7_Pin ExRAM_A7{BJ43}
BJ42 PPU_Fetch_Mux_8{M1} PPU_A8_Pin ExRAM_A8{BJ42}
BJ41 PPU_Fetch_Mux_9{M1} PPU_A9_Pin ExRAM_A9{BJ41}
NAND Mux 3

BJ40
CPU_A0_Pin
CPU_A1_Pin
CPU_A2_Pin
CPU_A3_Pin
CPU_A4_Pin
CPU_A5_Pin
CPU_A6_Pin
CPU_A7_Pin
CPU_A8_Pin
CPU_A9_Pin
Reg 10 D-Latch

/Q{BL109}
/CPU_Write_To_ExRAM_Any_Mode{BK85}
+ExRAM_Mode_$0x{BL96}
BK89
BK90

MMC5 Only
+PPU_RD_A_PIN{PinInv}
MMC5A Only
/Q{BL99}

BK78
BK77
CPU_D0_IN_PIN
CPU_D1_IN_PIN
CPU_D2_IN_PIN
CPU_D3_IN_PIN
CPU_D4_IN_PIN
CPU_D5_IN_PIN
CPU_D6_IN_PIN
CPU_D7_IN_PIN
CPU_D0_In_Latched{BK77}
CPU_D1_In_Latched{BK77}
CPU_D2_In_Latched{BK77}
CPU_D3_In_Latched{BK77}
CPU_D4_In_Latched{BK77}
CPU_D5_In_Latched{BK77}
CPU_D6_In_Latched{BK77}
CPU_D7_In_Latched{BK77}
Reg 8 D-Latch
Reg 8 D-Latch

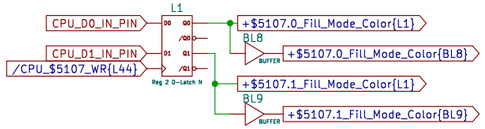
/Scanline_Counter_Reset{Z25}
+ExRAM_Mode_$0x{BL96}
BL95
BL104
BL103
BJ39b
Inverter
Buffer

BJ39 CPU_D0_In_Latched{BK77} PPU_D0_In{BigBuf} ExRAM_D0_Write{BJ39}
BJ38 CPU_D1_In_Latched{BK77} PPU_D1_In{BigBuf} ExRAM_D1_Write{BJ38}
BJ37 CPU_D2_In_Latched{BK77} PPU_D2_In{BigBuf} ExRAM_D2_Write{BJ37}
BJ36 CPU_D3_In_Latched{BK77} PPU_D3_In{BigBuf} ExRAM_D3_Write{BJ36}
BJ35 CPU_D4_In_Latched{BK77} PPU_D4_In{BigBuf} ExRAM_D4_Write{BJ35}
BJ34 CPU_D5_In_Latched{BK77} PPU_D5_In{BigBuf} ExRAM_D5_Write{BJ34}
BJ33 CPU_D6_In_Latched{BK77} PPU_D6_In{BigBuf} ExRAM_D6_Write{BJ33}
BJ32 CPU_D7_In_Latched{BK77} PPU_D7_In{BigBuf} ExRAM_D7_Write{BJ32}
NAND Mux 2

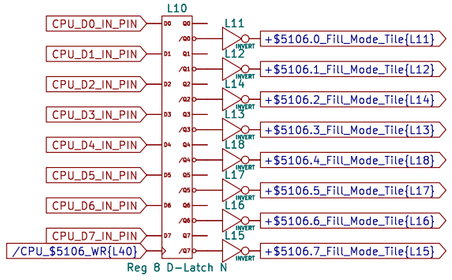
ExRAM_D0_Read
ExRAM_D1_Read
ExRAM_D2_Read
ExRAM_D3_Read
ExRAM_D4_Read
ExRAM_D5_Read
ExRAM_D6_Read
ExRAM_D7_Read
BJ12(a) ExRAM_D0_Read{BJ12(a)}
BJ12(b) ExRAM_D1_Read{BJ12(b)}
BJ12(c) ExRAM_D2_Read{BJ12(c)}
BJ12(d) ExRAM_D3_Read{BJ12(d)}
BJ12(e) ExRAM_D4_Read{BJ12(e)}
BJ12(f) ExRAM_D5_Read{BJ12(f)}
BJ12(g) ExRAM_D6_Read{BJ12(g)}
BJ12(h) ExRAM_D7_Read{BJ12(h)}
BUFFER
BJ11
/ExRAM_Read_D0_Latch{BJ11}
/ExRAM_Read_D1_Latch{BJ11}
/ExRAM_Read_D2_Latch{BJ11}
/ExRAM_Read_D3_Latch{BJ11}
/ExRAM_Read_D4_Latch{BJ11}
/ExRAM_Read_D5_Latch{BJ11}
+ExRAM_Read_D6_Latch{BJ11}
+ExRAM_Read_D7_Latch{BJ11}
+PPU_RD_A{BJ8}
+PPU_Fetch_Count.1{S13}
+PPU_Fetch_Count.0{S10}
BJ9
BJ10
BJ7
Reg 8 D-Latch N

Sheet: /ExRAM Mode/
File: exram_mode.kicad_sch
Title:
Size: USLedger | Date:
KiCad E.D.A. 9.0.7
Rev:
Id: 14/26

$5107 Register
Fill-Mode Color



$5106 Register
Fill-Mode Tile



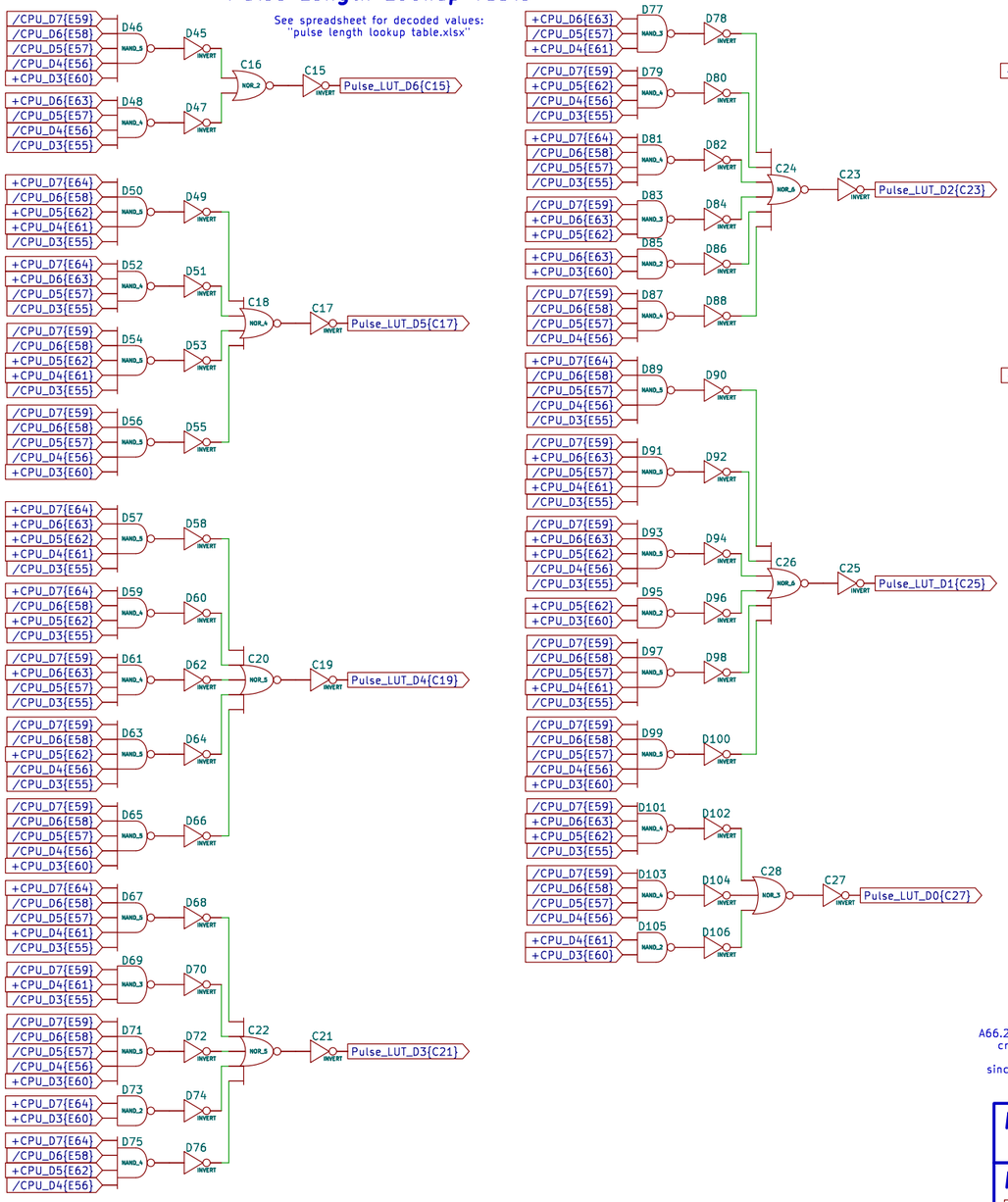
Sheet: /Fill Mode/
File: fill_mode.kicad_sch

Title:

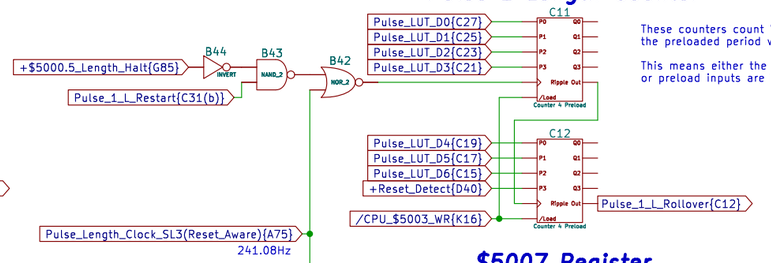
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 15/26

5-Bit to 7-Bit Pulse Length Lookup Table

See spreadsheet for decoded values: "pulse length lookup table.xlsx"



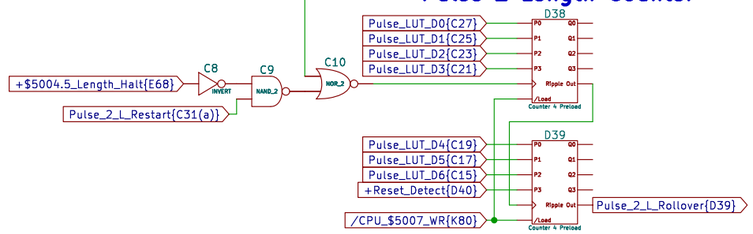
$5003 Register Pulse 1 Length Counter



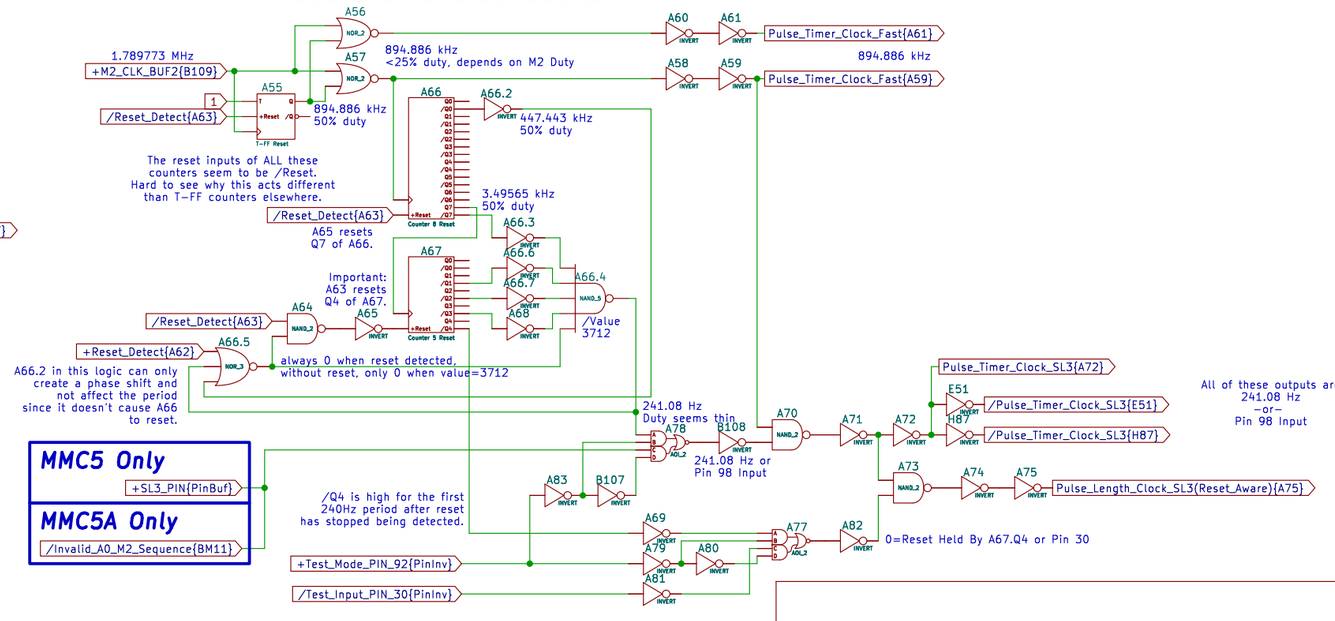
These counters count "down" the preloaded period value.

This means either the outputs or preload inputs are inverted.

$5007 Register Pulse 2 Length Counter



Pulse Channel Clock Source

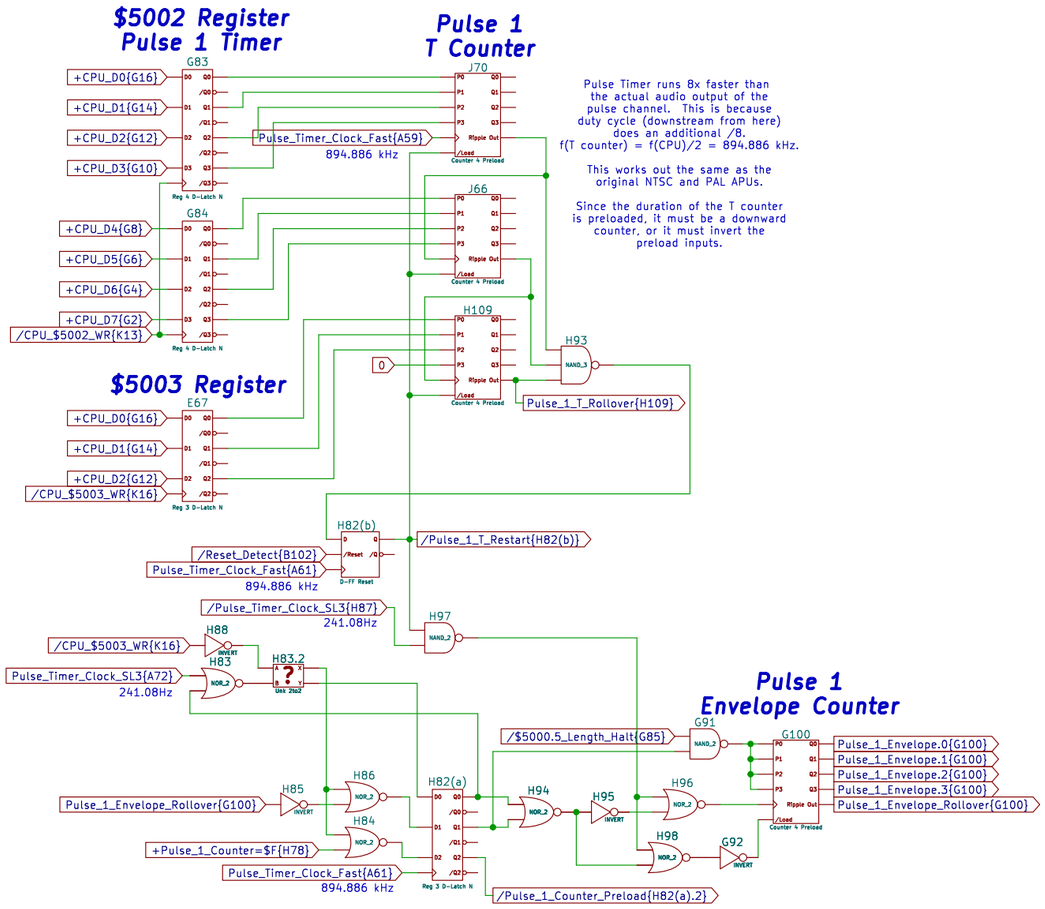


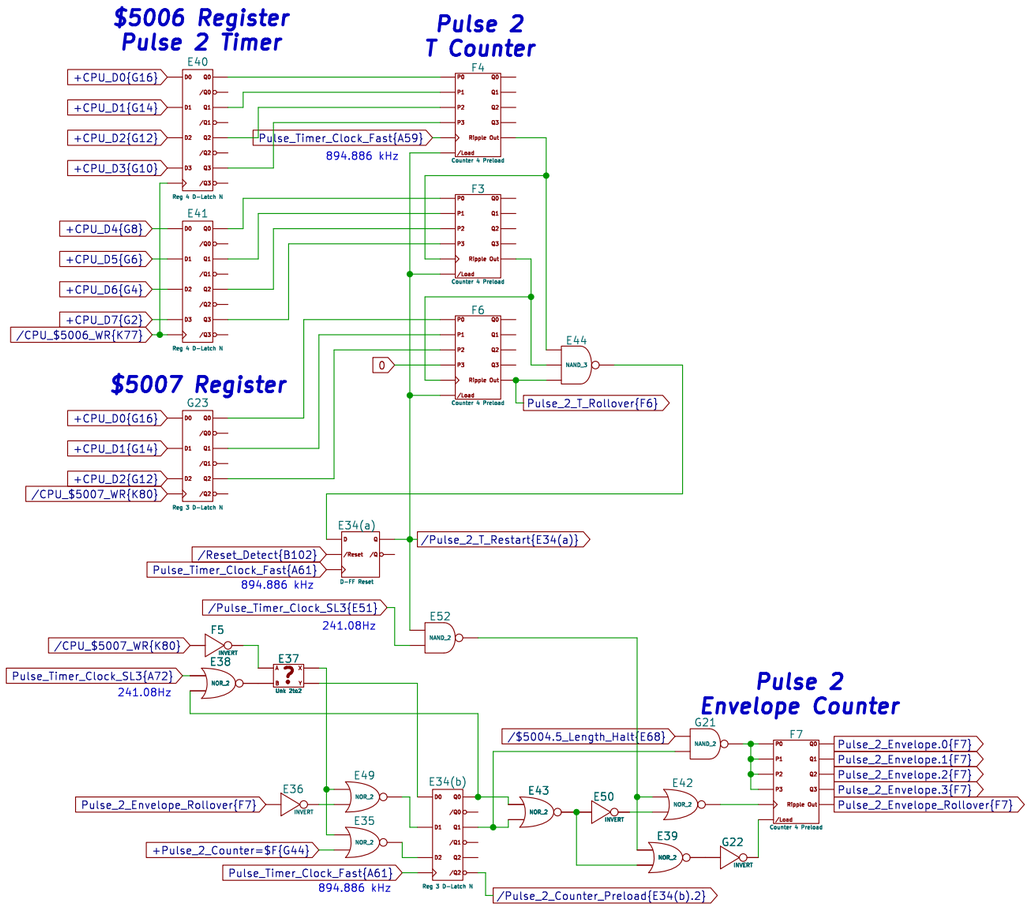
Sheet: /Pulse Length Lookup Table/
File: pulse_length_lookup_table.kicad_sch

Title:

Size: USLedger | Date: | **Rev:**

KiCad E.D.A. 9.0.7 | Id: 16/26



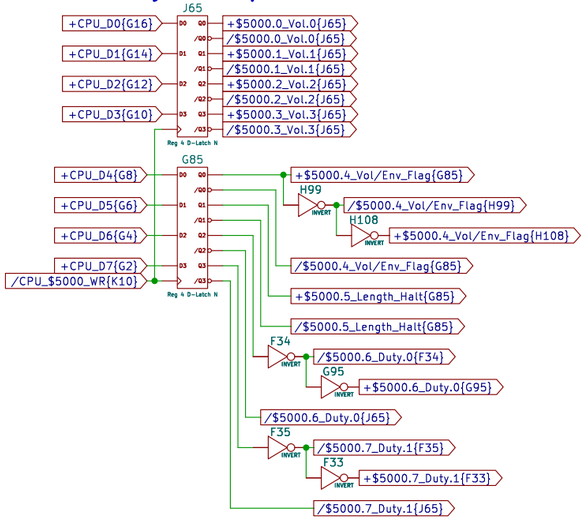


Sheet: /Pulse Timer/
File: pulse_timer.kicad_sch

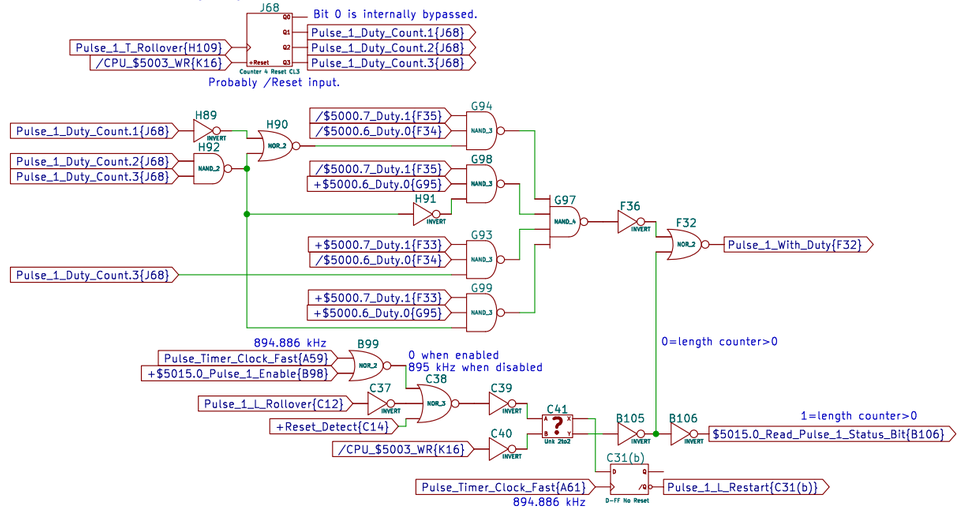
Title:

Size: USLedger | Date: | **Rev:**
KiCad E.D.A. 9.0.7 | Id: 17/26

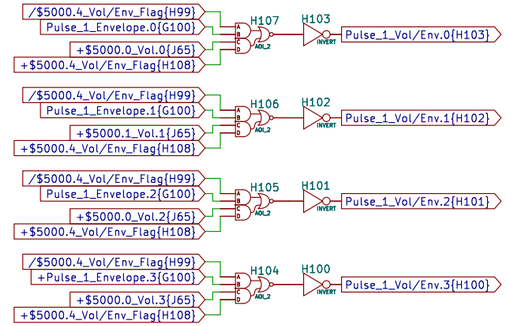
$5000 Register
Pulse 1 Duty Envelope Volume



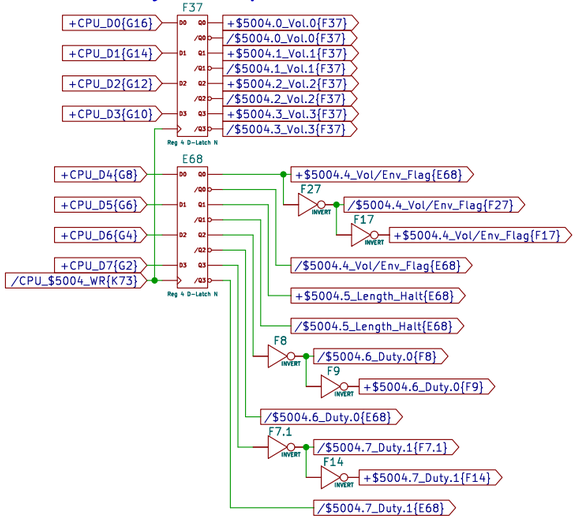
Pulse 1
Duty Cycle Counter



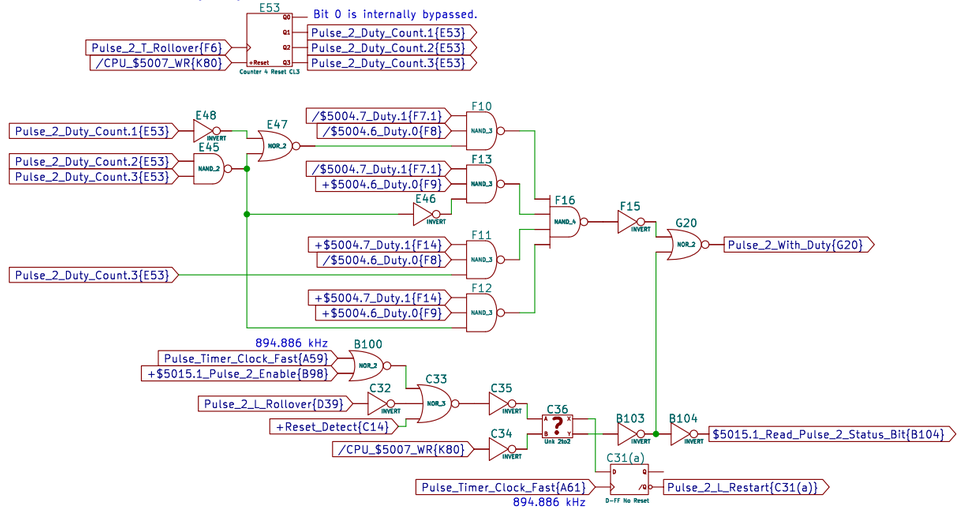
Pulse 1
Envelope/Volume Multiplexing



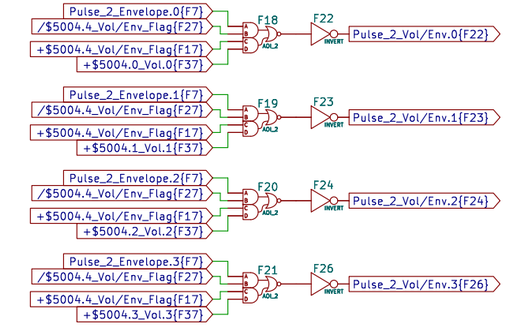
$5004 Register
Pulse 2 Duty Envelope Volume



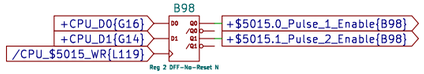
Pulse 2
Duty Cycle Counter



Pulse 2
Envelope/Volume Multiplexing



$5015 Write Register
Pulse 1&2 Status

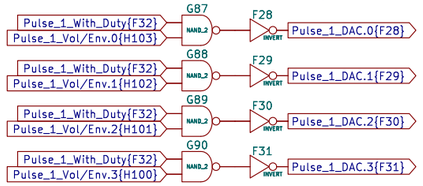


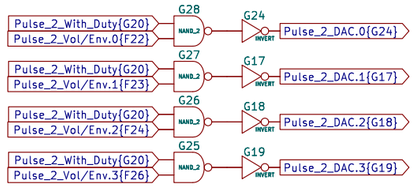
Sheet: /Pulse Duty Envelope Volume/
File: pulse_duty_envelope_volume.kicad_sch

Title:

Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 18/26

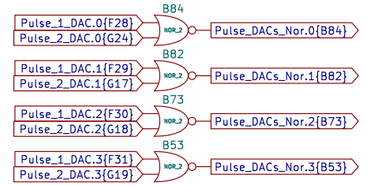
Pulse 1&2
Individual 4-bit DAC Outputs

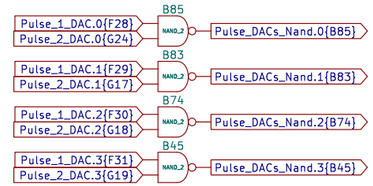




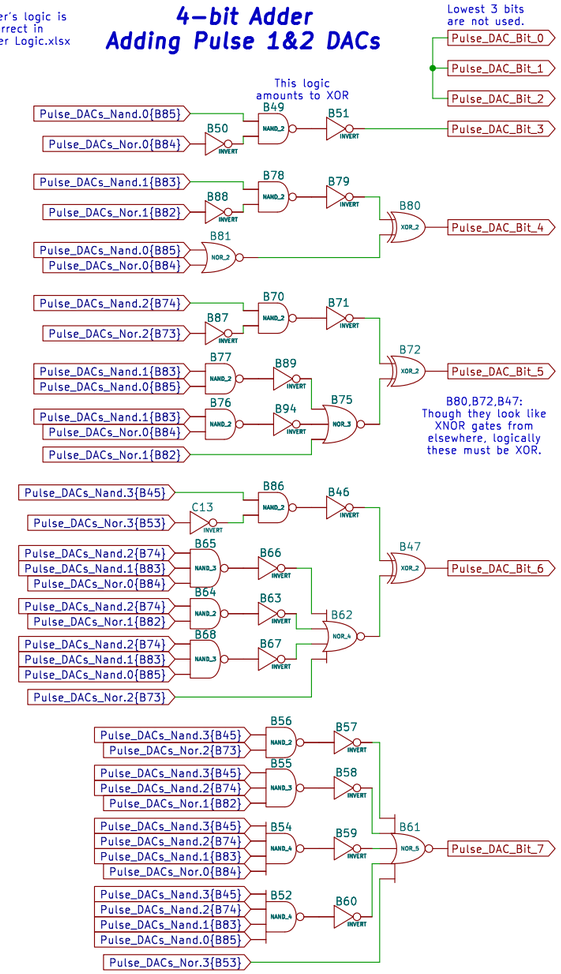
Components for
4-bit Adder

Note: The adder's logic is verified correct in Pulse DAC Adder Logic.xlsx





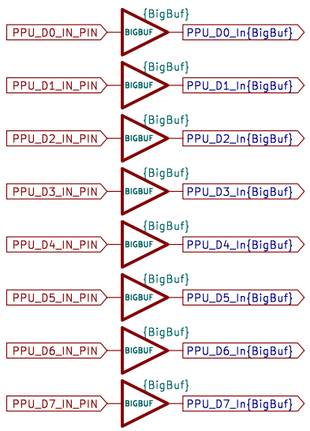
4-bit Adder
Adding Pulse 1&2 DACs

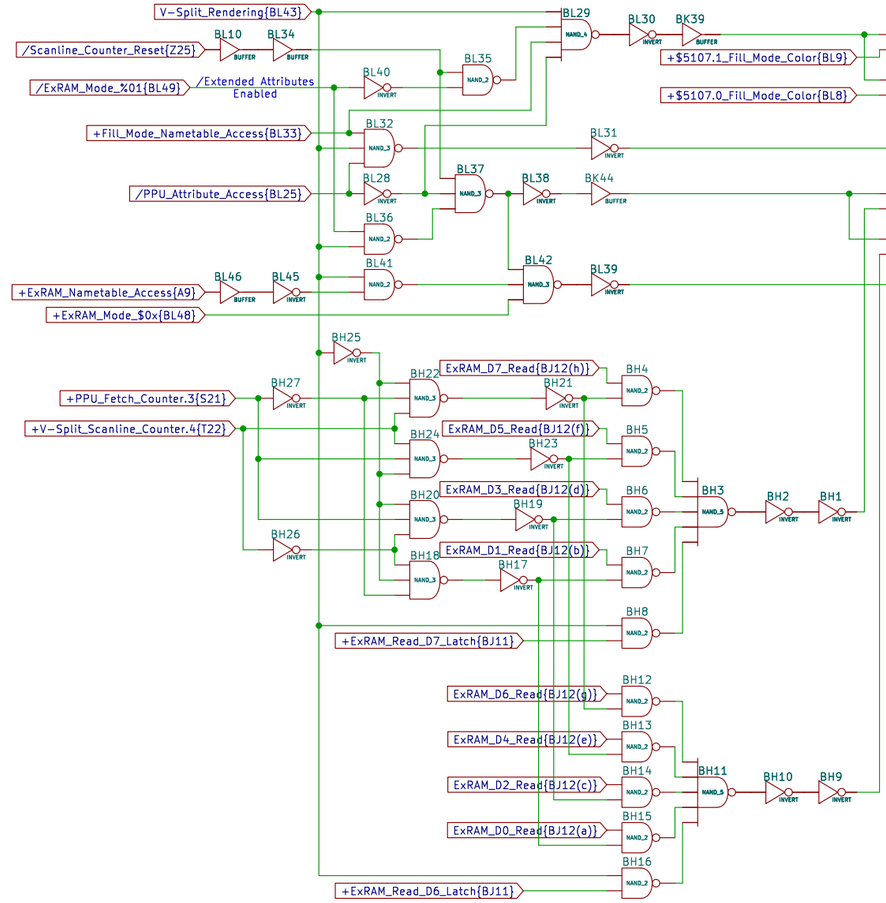


Sheet: /Pulse DAC/
File: Pulse DAC.kicad_sch

Title:

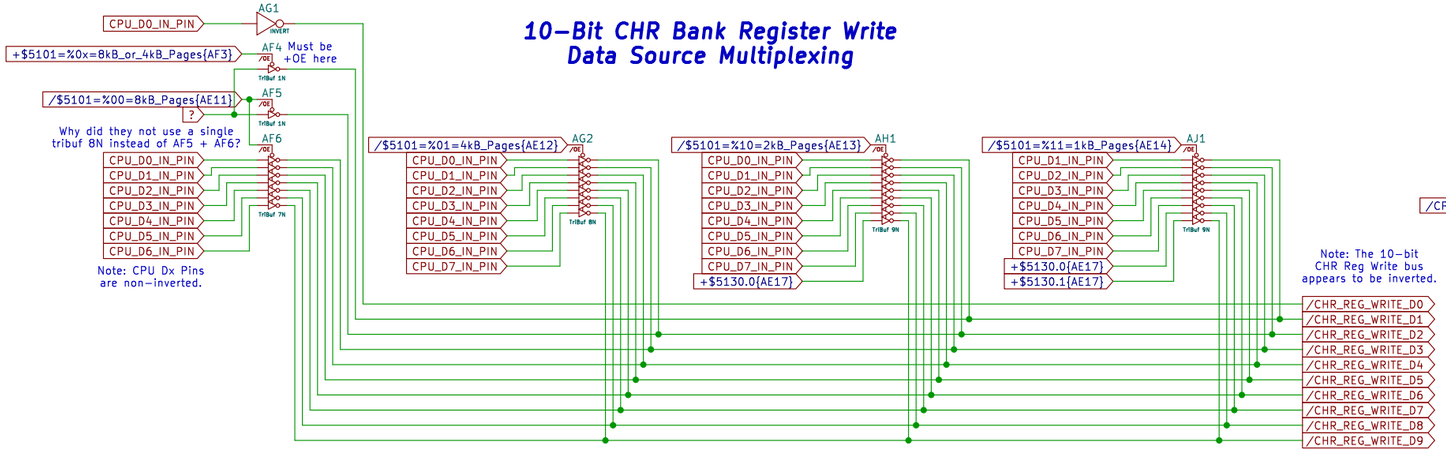
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 19/26



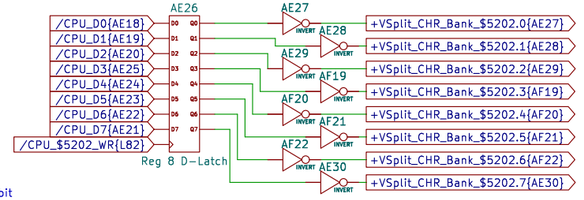


Sheet: /PPU Data Bus/		
File: ppu_data_bus.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 20/26

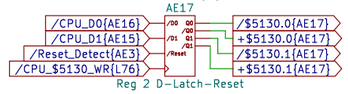
10-Bit CHR Bank Register Write Data Source Multiplexing



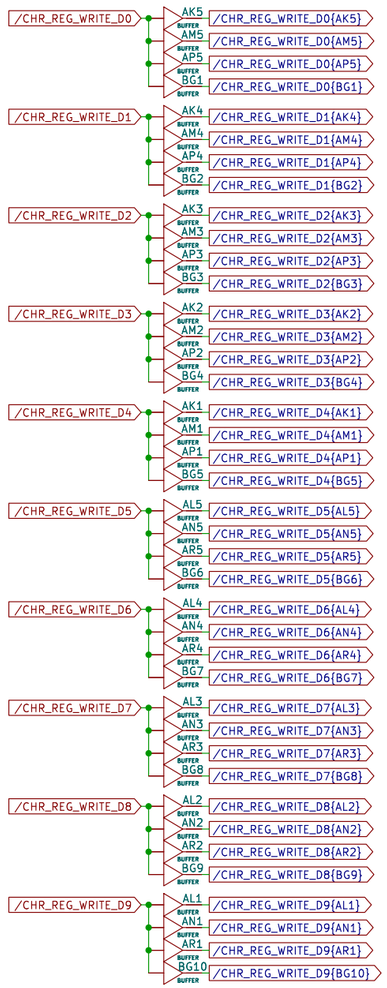
$5202 Register V-Split CHR Bank



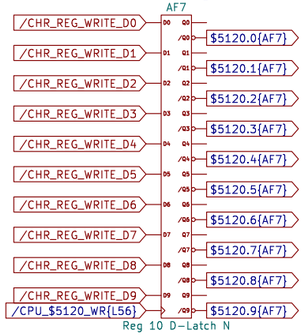
$5130 Register CHR Bank Upper Bits



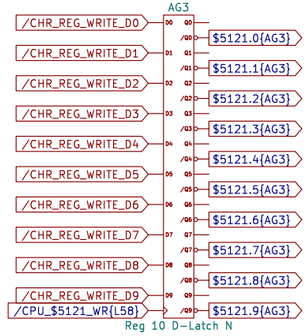
CHR Reg Write Buffering



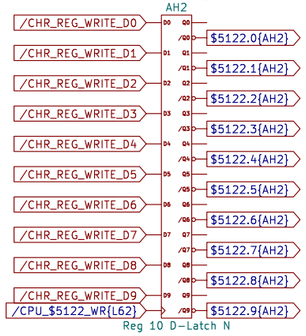
$5120 Register



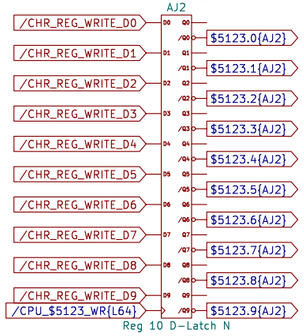
$5121 Register



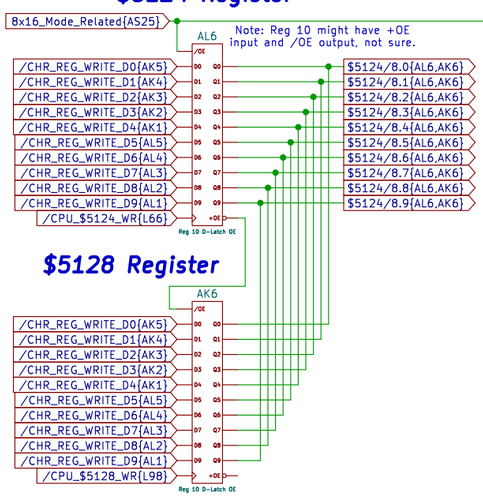
$5122 Register



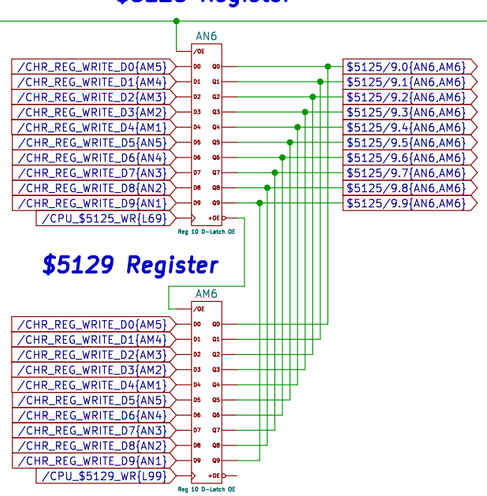
$5123 Register



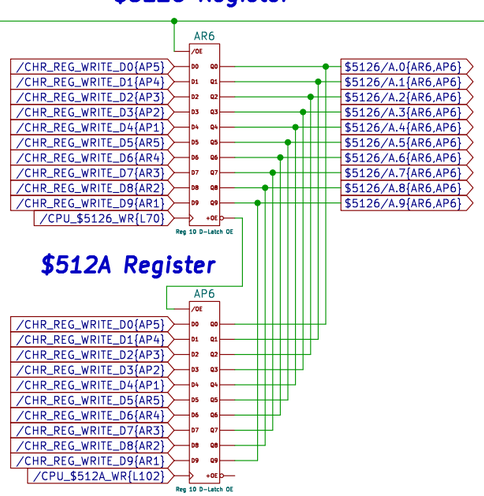
$5124 Register



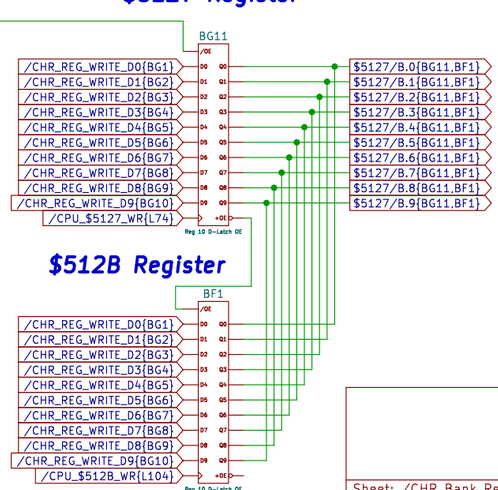
$5125 Register



$5126 Register



$5127 Register



$5128 Register

$5129 Register

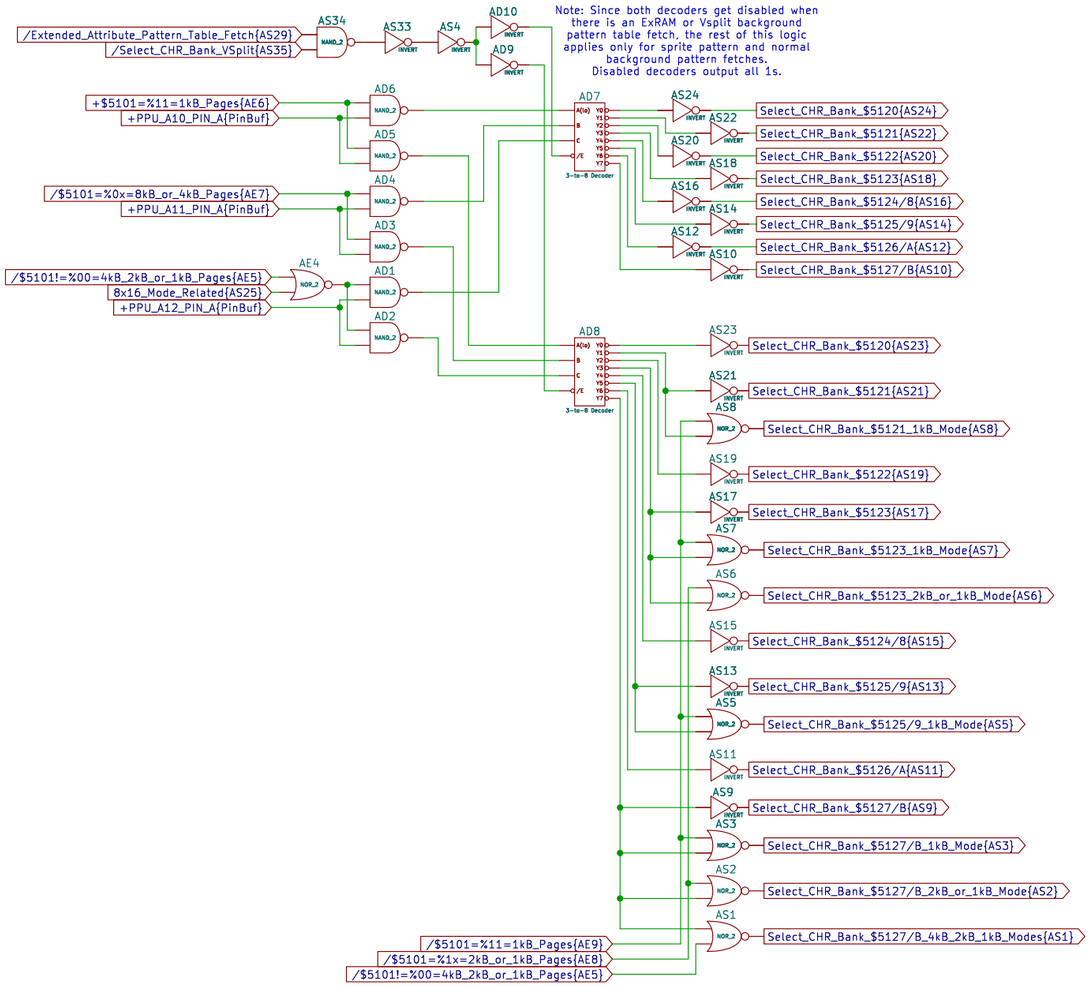
$512A Register

$512B Register

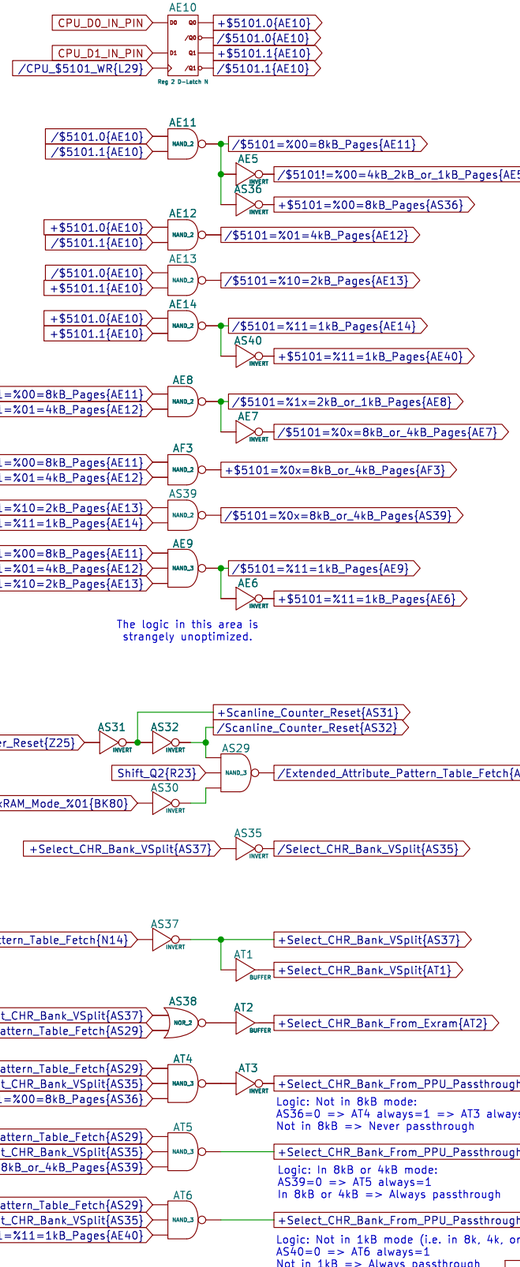
Sheet: /CHR Bank Registers/
File: chr_bank_registers.kicad_sch

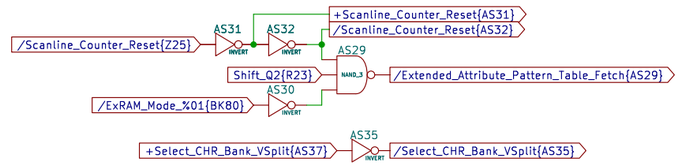
Title:

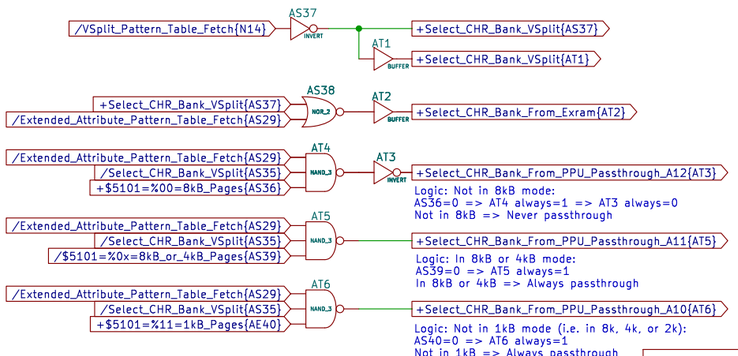
Size: USLedger | Date: | **Rev:**
KiCad E.D.A. 9.0.7 | Id: 21/26



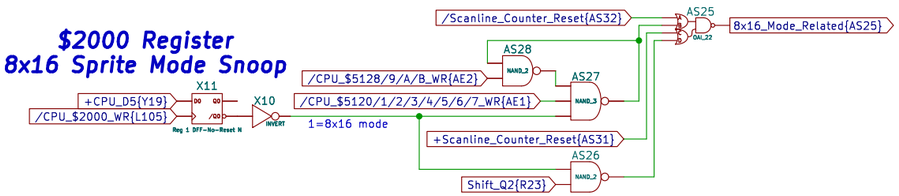
$5101 Register
CHR Mode





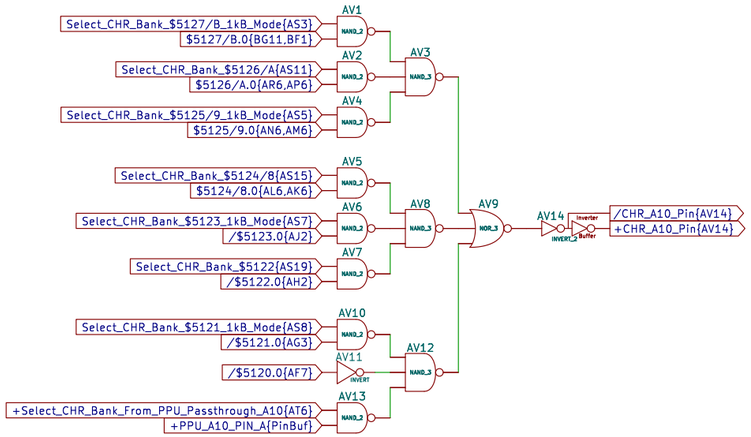


$2000 Register
8x16 Sprite Mode Snoop

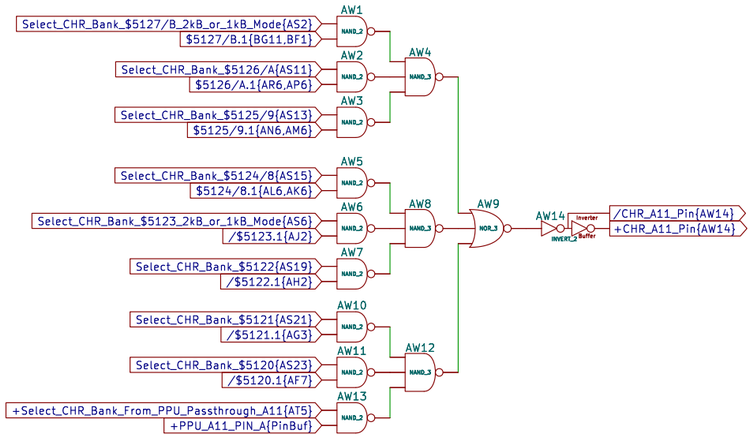


Sheet: /CHR Bank Selection Logic/
File: chr_bank_selection_logic.kicad_sch
Title:
Size: USLedger | Date: | Rev:
KiCad E.D.A. 9.0.7 | Id: 22/26

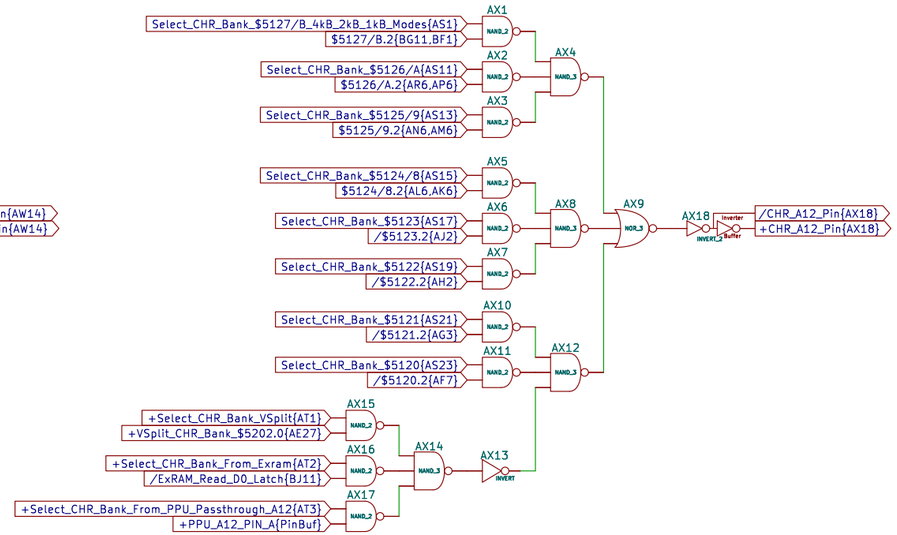
CHR A10



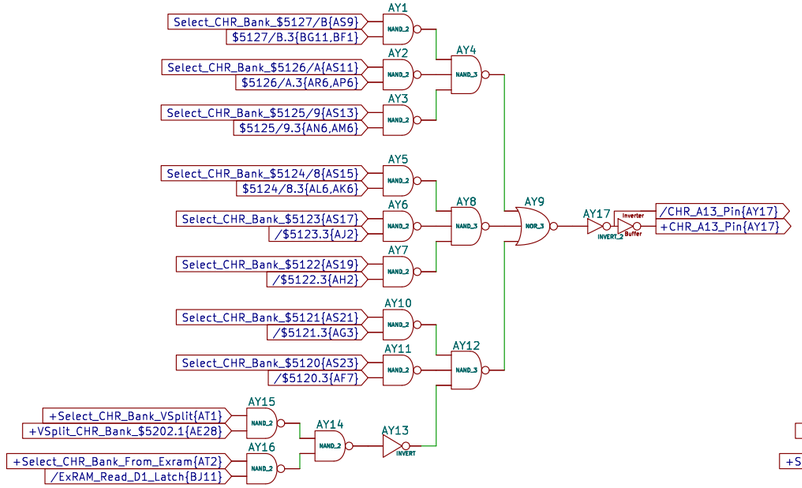
CHR A11



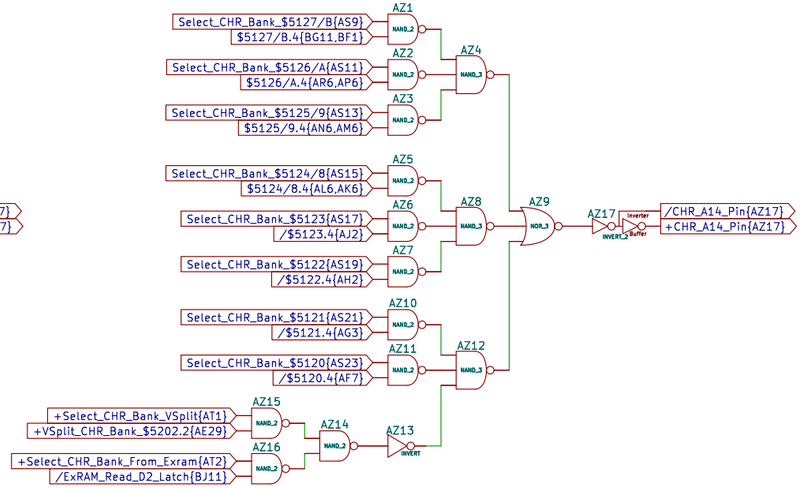
CHR A12



CHR A13



CHR A14

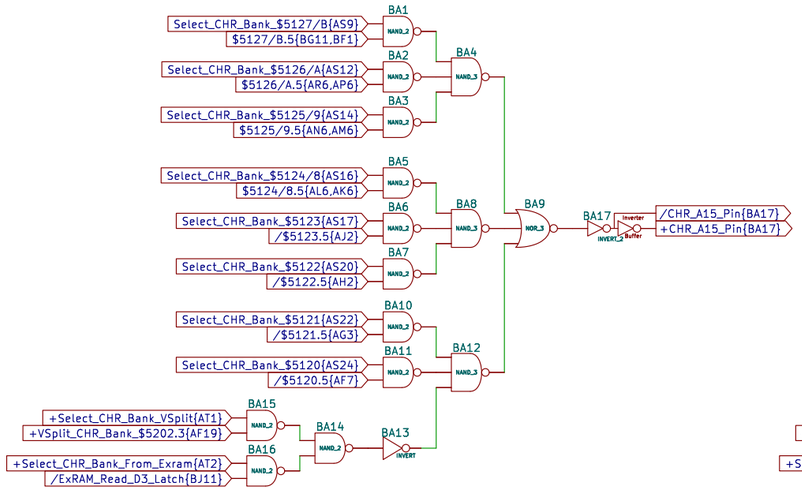


Sheet: /CHR Address Bus 10 to 14/
File: chr_address_bus_10_to_14.kicad_sch

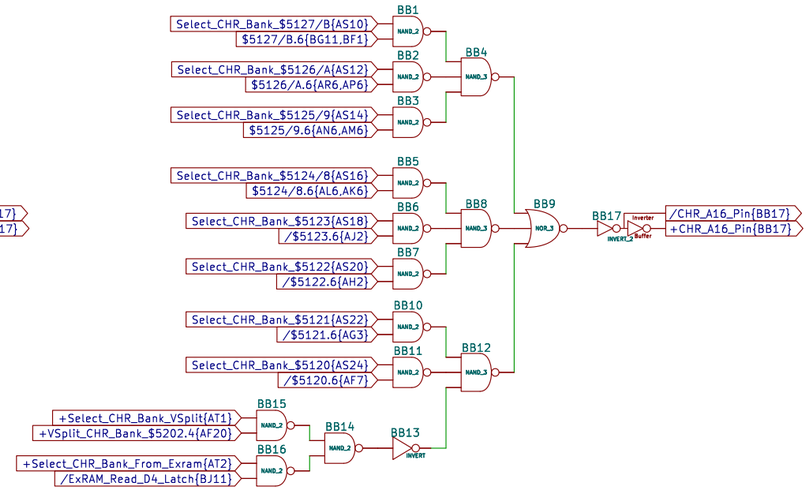
Title:

Size: USLedger | Date: | **Rev:**
KiCad E.D.A. 9.0.7 | Id: 23/26

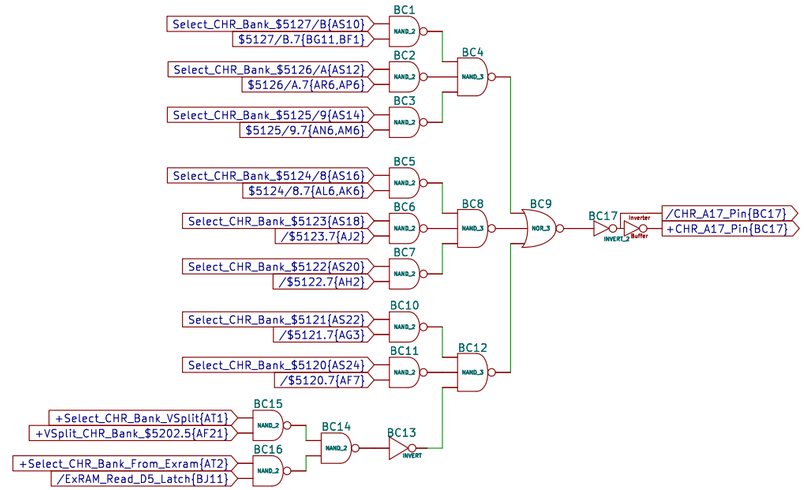
CHR A15



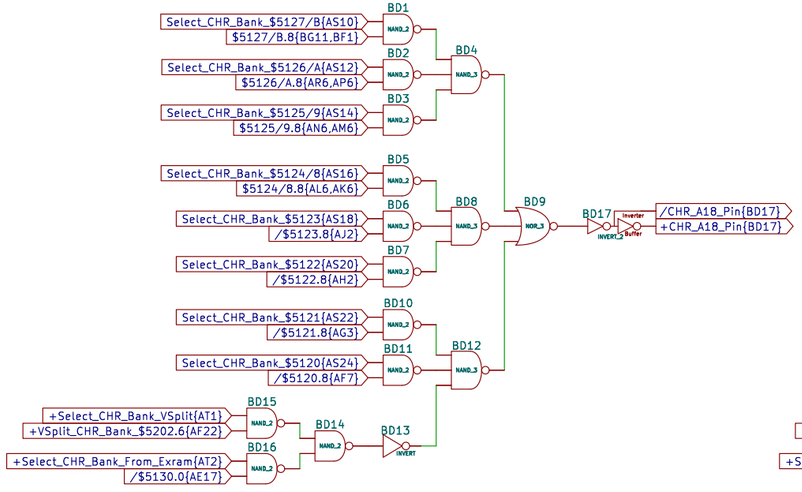
CHR A16



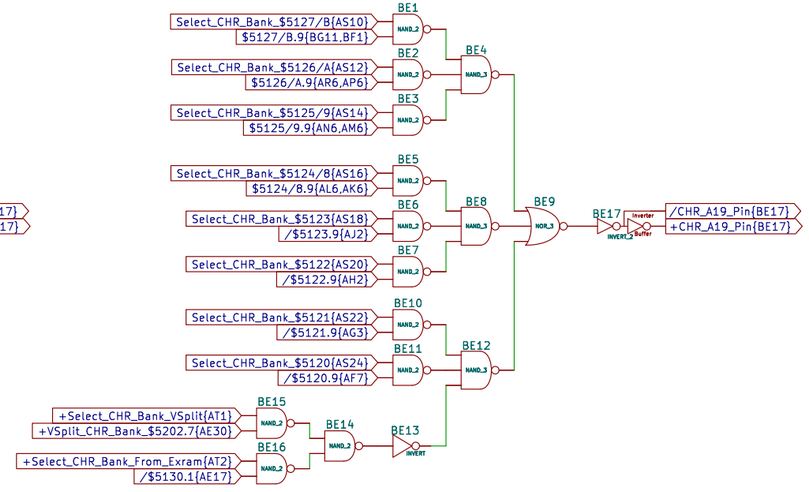
CHR A17



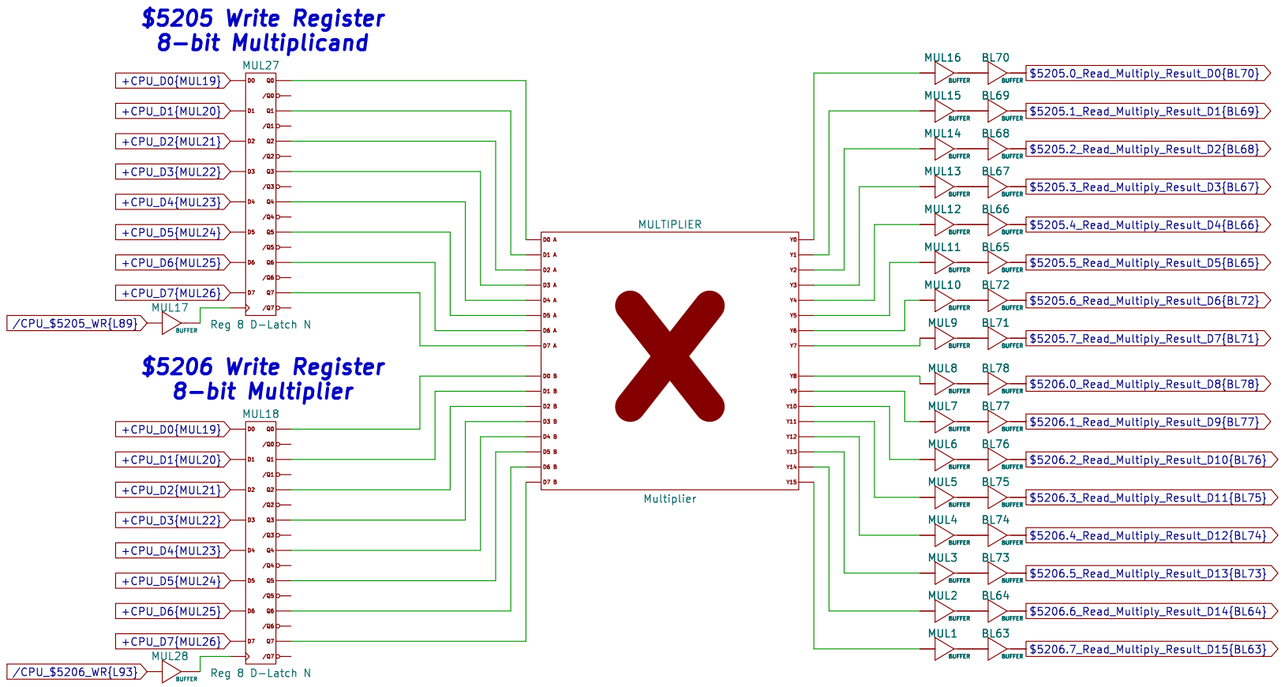
CHR A18



CHR A19



Sheet: /CHR Address Bus 15 to 19/		
File: chr_address_bus_15_to_19.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 24/26

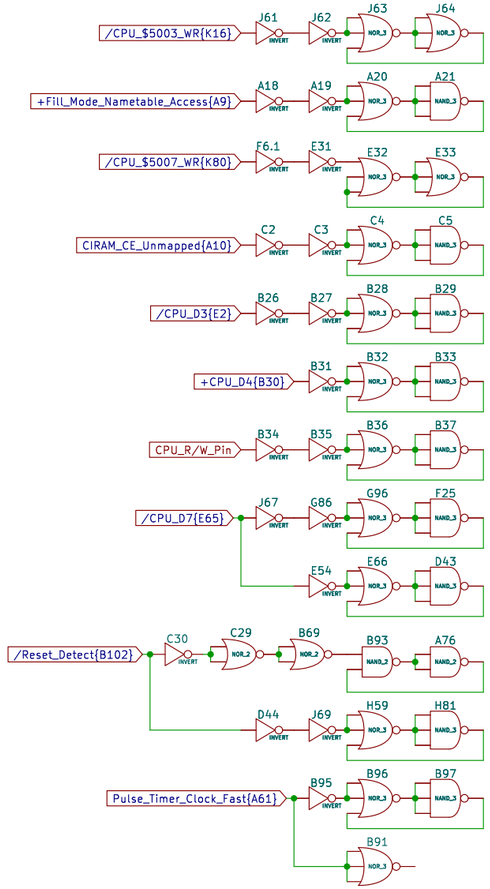


Sheet: /Multiplier/
File: multiplier.kicad_sch

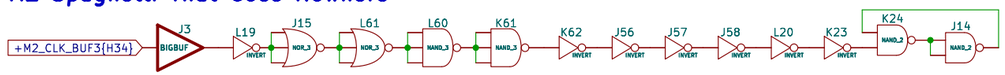
Title:

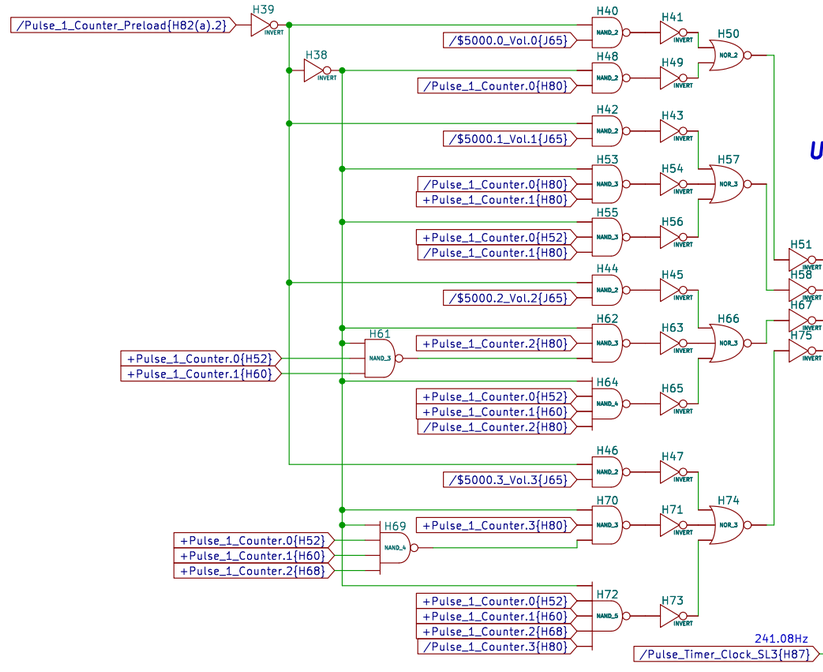
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 25/26

This sheet contains all residual items that don't serve any apparent purpose.



M2 Spaghetti That Goes Nowhere

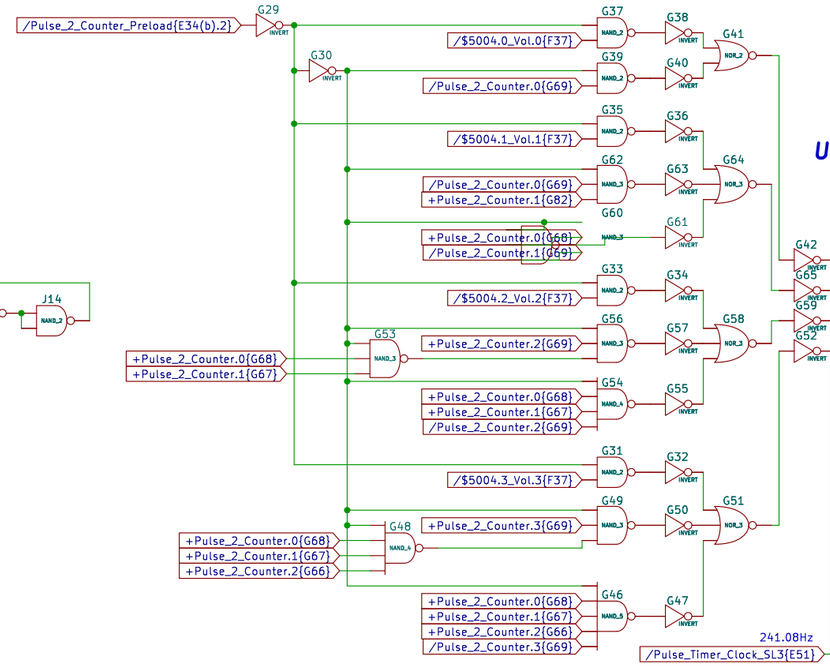




Pulse 1 Unused 4-bit Binary Up-Counter With Preload

The only output that goes elsewhere is H78, which gets latched by H82(a), coming directly back here as /Pulse_1_Counter_Preload; all self-contained. It does not appear to provide any other external signals, so I don't think it actually does anything. It seems like an abandoned envelope counter that was left in.

It latches the 4-bit pulse volume, then counts up at a rate of 240 Hz. Nothing stops it from counting or rolling over from $F back to $0.



Pulse 2 Unused 4-bit Binary Up-Counter With Preload

Sheet: /Artifacts/		
File: artifacts.kicad_sch		
Title:		
Size: USLedger	Date:	**Rev:**
KiCad E.D.A. 9.0.7		Id: 26/26